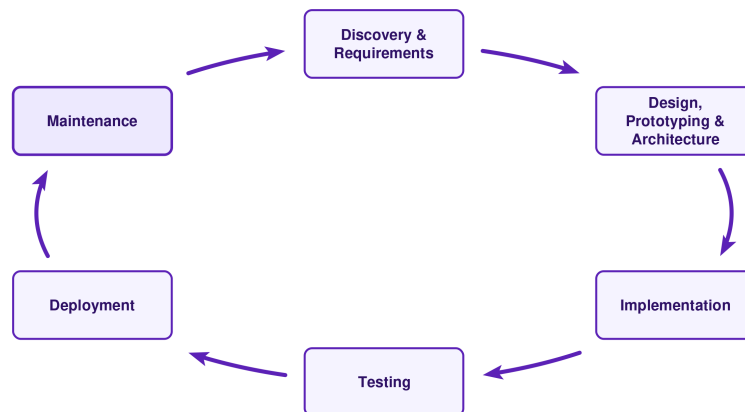




A Process Framework for Human-AI Collaboration in the Software Development Lifecycle

Design and Instantiation of a Supporting Tool

Tomás dos Santos Taborda

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisors: Prof. Miguel Leitão Bignolas Mira da Silva
Hugo de Sousa

October 2026

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa. I acknowledge the use of artificial intelligence tools to support the drafting and refinement of this thesis, with all content produced by these tools carefully reviewed, edited, and validated to ensure accuracy and originality.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetur erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis.

Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus.

Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you - Thank you.

Abstract

Generative Artificial Intelligence and Large Language Models have moved from autocomplete assistants to active contributors across the Software Development Life Cycle, enabling development in which practitioners express requirements in natural language and supervise the resulting artefacts. Adoption, however, has outpaced method. A Systematic Literature Review over 646 articles confirms substantial benefits alongside recurring risks in reliability, security, traceability and accountability, and identifies the absence of structured, repeatable guidance for integrating these tools into professional workflows as the field's central unresolved problem. A majority of functionally correct AI-generated code has been reported to carry security defects, and delivery bottlenecks shift towards review rather than disappearing: value is conditional on process, not on model capability alone.

This dissertation addresses that gap through the Design Science Research Methodology. It proposes a process-oriented framework for governed Human-AI collaboration treating AI systems as first-class collaborators whose contributions must be bounded, validated and traced. The framework defines lifecycle phases with phase-specific quality gates, responsibilities expressed as functions rather than job titles, and governance metrics, anchored by a persistent versioned specification linking every generated artefact to its originating requirement. It is instantiated as Apex, a web application operationalising each phase, gate and artefact against a real project management backend. Evaluation combines application in a real project, standardised usability and workload instruments, interviews with practitioners and Agile experts, and an analytical assessment against criteria set in advance. The contribution is an evidence-grounded process model allowing organisations to adopt AI assistance without surrendering oversight of software quality, security and accountability.

Keywords

Human-AI Collaboration; Software Development Life Cycle; Large Language Models; Process Framework; AI Governance; Design Science Research

Resumo

A Inteligência Artificial Generativa e os Modelos de Linguagem de Grande Dimensão passaram de assistentes de preenchimento automático a contribuidores activos no Ciclo de Vida de Desenvolvimento de Software, permitindo exprimir requisitos em linguagem natural e supervisionar os artefactos gerados. A adopção, contudo, ultrapassou o método. Uma Revisão Sistemática da Literatura sobre 646 artigos confirma benefícios substanciais a par de riscos recorrentes de fiabilidade, segurança, rastreabilidade e responsabilização, e identifica a ausência de orientação estruturada e repetível para integrar estas ferramentas em contextos profissionais como o principal problema por resolver. A maioria do código gerado por IA funcionalmente correcto contém defeitos de segurança, e os estrangulamentos deslocam-se para a revisão em vez de desaparecerem: o valor depende do processo, não do modelo.

Esta dissertação aborda essa lacuna através da Design Science Research Methodology, propondo uma framework orientada a processo para a colaboração Humano-IA governada, cujas contribuições de IA devem ser delimitadas, validadas e rastreadas. Define fases com portões de qualidade específicos, responsabilidades expressas como funções e não como cargos, e métricas de governação, ancoradas numa especificação versionada que liga cada artefacto ao seu requisito. É instanciada no Apex, aplicação web que operacionaliza cada fase, portão e artefacto sobre uma ferramenta real de gestão de projectos. A avaliação combina aplicação num projecto real, instrumentos padronizados de usabilidade e carga cognitiva, entrevistas, e avaliação analítica face a critérios previamente definidos. O contributo é um modelo de processo fundamentado em evidência que permite adoptar assistência por IA sem abdicar do controlo sobre qualidade, segurança e responsabilização.

Palavras Chave

Colaboração Humano-IA; Ciclo de Vida de Desenvolvimento de Software; Modelos de Linguagem de Grande Dimensão; Framework de Processo; Governação de IA; Design Science Research

Contents

1	Introduction	1
2	Research Methodology	3
2.1	Systematic Literature Review (SLR)	3
2.2	Design Science Research Methodology (DSRM)	4
2.3	Mapping the Methodology onto this Document	5
3	Research Background	6
3.1	Artificial Intelligence	6
3.2	Software Development Life Cycle	7
3.3	AI in Software Development	8
3.4	Vibe Coding	8
3.5	Spec-Driven Development	8
4	Systematic Literature Review	10
4.1	Planning the Review	10
4.1.1	Motivation	10
4.1.2	Research Questions	11
4.1.3	Review Protocol	11
4.2	Conducting the Review	12
4.2.1	Filtering Process	12
4.2.2	Data Extraction Analysis	13
4.3	Reporting the Review	13
4.3.1	Benefits of AI and LLMs usage in Software Development	14
4.3.2	Consequences of AI and LLMs usage in Software Development	16
4.3.3	Methodologies and management methods used to integrate AI and LLMs in Software Development	17
4.3.4	Developers' and other stakeholders' opinions regarding the use of AI and LLMs in Software Development	19
4.3.5	Industry sectors adopting AI and LLMs for Software Applications Development	20
4.4	Discussion	21
5	Research Problem	23
6	Research Proposal	25
6.1	Objectives	26
6.2	Purpose and Scope	26
6.3	Design Principles	27
6.4	Lifecycle Phases and the AI Interaction Mapping	28
6.5	Roles and Responsibilities	30
6.6	Governance Mechanisms and Quality Gates	32
6.6.1	Bolts as the Unit of Execution	32

6.6.2	Spec-Anchored Continuity	33
6.6.3	Quality Gates	33
6.6.4	Governance Metrics	34
6.7	Work Organisation	35
6.8	Operational Playbooks	35
6.9	Positioning Against Alternatives	37
6.10	Summary	38
7	Proposal Implementation	39
7.1	Purpose and Positioning	39
7.2	Architecture	40
7.3	Operationalising the Framework	44
7.4	Design History and Practitioner Feedback	48
7.5	Quality Assurance of the Implementation	50
7.6	Limitations of the Implementation	51
8	Demonstration	52
8.1	Demonstration Design	52
8.2	Context	55
8.3	Application of the Framework	55
8.4	Observations	56
8.5	Summary	56
9	Evaluation	57
9.1	Evaluation Strategy	57
9.2	Participants	59
9.3	Instruments and Procedure	59
9.3.1	System Usability Scale	60
9.3.2	NASA Task Load Index	60
9.3.3	Apex UX Questionnaire	61
9.3.4	Semi-Structured Interviews	61
9.3.5	Analytical Assessment	63
9.4	Results	64
9.5	Threats to Validity	64
9.6	Discussion	64
9.6.1	Answering the Research Questions	65
10	Conclusion	66
10.1	Summary and Contributions	66
10.2	Communication	66
10.3	Limitations	67
10.4	Future Work	67
	Declaration of Generative AI and AI-Assisted Technologies in the Writing Process	68
	Bibliography	68
A	Extended Results for the Systematic Literature Review	74
A.1	Distribution of the retained corpus by venue	74
A.2	Stakeholder opinions reported in the literature	74
A.3	Industry sectors adopting AI and LLMs	76
A.4	Final set of studies retained by the review	77

B	Evaluation Instruments	81
B.1	Informed consent and demographics	81
B.2	Raw NASA Task Load Index	82
B.3	System Usability Scale	83
B.4	Apex UX Questionnaire	84
B.5	Interview guides	85
B.6	Per-participant raw scores	86

List of Figures

2.1	Three phases of the Systematic Literature Review	4
2.2	Phases of the Design Science Research Methodology	4
4.1	Articles Filtering Process	12
4.2	Academic Journal Publications	14
6.1	The six lifecycle phases in sequence, and the three named loop-backs out of Maintenance. Each loop-back re-enters a specific earlier phase, whose gate then applies in full - Maintenance never patches production code without a corresponding change to the specification.	29
7.1	Apex's system architecture, part one of two: the frontend, the backend and the specification store. The figure is split across two pages for legibility; the point of it is the position of two boundaries rather than the inventory of components, and this half shows the first, where all workflow state is written to a per-tenant, per-project specification store rather than to an application database. The backend's guarded egress layer is drawn collapsed to a single component here and expanded in Figure 7.2; the dashed connector on the right marks where the two halves join.	41
7.2	Apex's system architecture, part two of two, expanding the guarded egress layer of Figure 7.1 and the external services it reaches. This half shows the second boundary the figure exists to make, that every outbound call to a user-configurable host passes through a server-side guarded proxy. The dashed connectors carry over the callers from Figure 7.1; the browser-direct path to the version control host is the single documented exception, and the credential for it never reaches the backend.	42
7.3	A representative walk through Apex, Phases 1 to 3. The figure is split across two pages for legibility; what it is meant to show is that every phase terminates in a write to the specification store and a human acceptance, so no phase can be exited by generation alone.	46
7.4	The same walk, Phases 4 to 6, continuing Figure 7.3. Note the loop-backs out of Testing and out of Maintenance: neither returns to an undefined state, and each re-enters a named phase whose gate then applies in full.	47

List of Tables

2.1	Mapping between DSRM activities and the chapters of this dissertation	5
4.1	Research Questions and Focus Areas	11
4.2	Research Elements and Details	12
4.3	Inclusion and Exclusion Criteria	13
4.4	Benefits of using AI and LLMs in Software Development	14
4.5	Consequences of using AI and LLMs in Software Development	16
4.6	Methodologies and Management Methods used to integrate AI and LLMs in Software Development	18
4.7	Impact of AI and LLMs across Software Development Lifecycle Phases	21
6.1	Mapping between Research Questions, Key Findings, Problem Aspects, and Framework Objectives	26
6.2	The eight design principles and the evidence grounding each	27
6.3	The six lifecycle phases and the preceding Strategic Alignment stage: permitted AI contribution, output, accountable hat, and the control that must be satisfied before each is exited. Each phase takes as input the output of the one above it.	30
6.4	The hats: the function each discharges and the gate each owns	31
6.5	The five quality gates: owner, required evidence, and the consequence of failure	34
6.6	The six operational playbooks: trigger, procedure, and termination	36
7.1	Framework constructs against the mechanisms implementing them in Apex, and the outcome of each	44
9.1	Evaluation instruments, target artefact, and research question addressed	58
A.1	Public / Researchers Opinions on AI and LLMs in Software Development	75
A.2	Developers / Practitioners Opinions on AI and LLMs in Software Development	75
A.3	Organizations / Management Opinions on AI and LLMs in Software Development	76
A.4	Industry sectors adopting AI and LLMs	77
A.5	Studies retained by the systematic literature review	78

Acronyms

AI	Artificial Intelligence
AI-DLC	AI-Driven Development Lifecycle
API	Application Programming Interface
BDD	Behaviour-Driven Development
CI	Continuous Integration
DRQ	Dissertation Research Question
DSRM	Design Science Research Methodology
GenAI	Generative Artificial Intelligence
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure as Code
IST	Instituto Superior Tecnico
KPI	Key Performance Indicator
LLM	Large Language Model
MLOps	Machine Learning Operations
NASA-TLX	NASA Task Load Index
NIST	National Institute of Standards and Technology
OKR	Objective and Key Result
PM	Project Manager
PO	Product Owner
QA	Quality Assurance
SDLC	Software Development Life Cycle
SLR	Systematic Literature Review
SUS	System Usability Scale
UI	User Interface
UUID	Universally Unique Identifier
UX	User Experience

1

Introduction

Generative Artificial Intelligence (GenAI) and Large Language Models (LLMs) have extended machine assistance towards generating coherent, context-aware artefacts, introducing intent-driven programming, colloquially *Vibe Coding*, in which developers describe desired behaviour in natural language and refine what a model produces [1, 2]. The shift is not confined to code: LLMs now participate in requirements elicitation, architectural exploration, test generation, documentation and debugging, touching every Software Development Life Cycle (SDLC) phase [3–5]. The practitioner’s work moves from constructing software towards specifying intent, supervising generated artefacts and remaining accountable for them.

The evidence on outcomes, however, is more nuanced than early enthusiasm suggested, and that gap motivates this work. Adoption is close to universal while trust is not: 84% of developers report using or planning to use AI tools, up from 76%, while the proportion trusting their accuracy fell from 40% to 29% [6], with daily use around 90% [7]. That distrust is warranted, and the decisive point is that functional correctness and security do not move together: 61% of a leading coding agent’s solutions were functionally correct but only 10.5% secure [8], and across more than one hundred models 45% of samples introduced an OWASP Top 10 vulnerability, with newer models improving at correctness while security stayed flat [9].

Productivity gains are equally conditional. Early evidence reported a task completed 55.8% faster with an assistant, though unpublished in a peer-reviewed venue [10], whereas a randomised controlled trial found 16 experienced developers on their own mature repositories took 19% longer while estimating afterwards that they had been 20% faster [11]; the favourable result came from a self-contained task with no codebase to respect. At organisational scale, telemetry across 1,255 teams associates high adoption with 98% more pull requests merged but 91% longer review times and no significant correlation with delivery outcomes [12]. The bottleneck moves downstream to validation rather than disappearing. Several

sources here are industry-reported rather than peer-reviewed and are identified as such throughout, so the argument rests on consistency of direction rather than any single figure: the value of AI assistance is conditional on the process surrounding it rather than a property of the model alone.

To characterise practice rigorously, an Systematic Literature Review (SLR) following Kitchenham's guidelines retrieved 646 articles, reported in Chapter 4. Its central finding is methodological: organisations lack established methodologies for the responsible use of Artificial Intelligence (AI) in professional software development [13–16], and proposed approaches remain fragmented across isolated tools, sharing no common foundation on lifecycle integration, oversight or evaluation [3]. Established methodologies do not close the gap: Waterfall, Scrum, Kanban and the Agile Manifesto were designed for environments in which every contributor is human [4], so they lack structures for governing AI participation, bounding the context a model may act upon, and allocating responsibility over generated artefacts, while their cadence assumes implementation is the binding constraint, which the evidence contradicts. **The research problem is therefore the absence of structured, repeatable guidance and standardised evaluation practices for integrating AI across the SDLC while preserving human oversight, software quality and security, and accountability.**

The objective is to design, instantiate, demonstrate and evaluate a process-oriented framework addressing that problem. Four questions structure it, labelled *Dissertation Research Question (DRQ)* to keep them distinct from the review questions of Chapter 4, which ask what the literature reports rather than what these artefacts achieve. **DRQ1:** how can AI-supported activities be mapped to SDLC phases so that the point, purpose and boundary of AI participation are unambiguous? **DRQ2:** which governance mechanisms, quality gates and human-in-the-loop practices keep AI contributions traceable, validated and accountable? **DRQ3:** to what extent can those constructs be operationalised in a working system, and which resist implementation? **DRQ4:** does the framework, through that implementation, give practitioners usable and cognitively sustainable support in a real setting? Chapters 6 to 9 answer them in turn, the third including the constructs weakened, proxied or left unimplemented.

The contributions are four: the synthesis above; a process framework treating AI as a first-class collaborator, with phases, gates, function-based responsibilities, metrics and a versioned specification anchoring artefacts to requirements; Apex, a reference implementation showing those constructs implementable rather than describable; and an empirical evaluation of its usability and workload.

2

Research Methodology

Contents

2.1 Systematic Literature Review (SLR)	3
2.2 Design Science Research Methodology (DSRM)	4
2.3 Mapping the Methodology onto this Document	5

This chapter presents the two research methodologies used in this dissertation, the SLR and the Design Science Research Methodology (DSRM), and maps the activities of the second onto the chapters of this document.

2.1 Systematic Literature Review (SLR)

The review follows Kitchenham's guidelines, which describe a systematic review as "a means of evaluating and interpreting all available research relevant to a particular research question, topic area, or phenomenon of interest" conducted through "a trustworthy, rigorous, and auditable methodology." The guidelines prescribe three phases: **planning**, which defines the motivation, research questions, selection criteria and protocol; **conducting**, which applies the protocol to identify and select studies; and **reporting**, which synthesises the findings.

Figure 2.1 illustrates those phases and their activities. Filtering ran over several iterations: an initial screening of retrieved articles, an abstract-reading pass to assess relevance, a filter applying Kitchenham's criteria to introductions and conclusions, and a full-text review determining the retained set. The

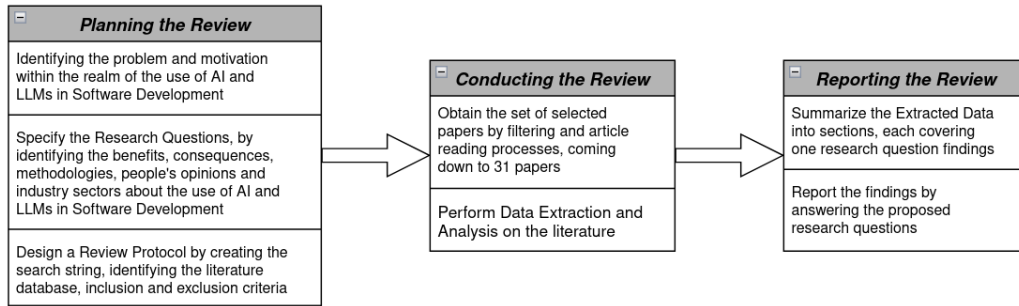


Figure 2.1: Three phases of the Systematic Literature Review

methodology was selected because it enables a structured and unbiased collection of existing research on AI-supported software development and its associated management practices.

2.2 Design Science Research Methodology (DSRM)

The *DSRM* concerns the development of IT artefacts, such as models, methods or frameworks, addressing identified organisational or technological problems. Unlike purely observational research it emphasises constructing solutions and evaluating their usefulness. Proposed by Peffers et al., it follows a six-step process supporting systematic design, clear contributions and transferable knowledge, which makes it suitable for research proposing process-oriented solutions. The steps are illustrated in Figure 2.2 and described below.

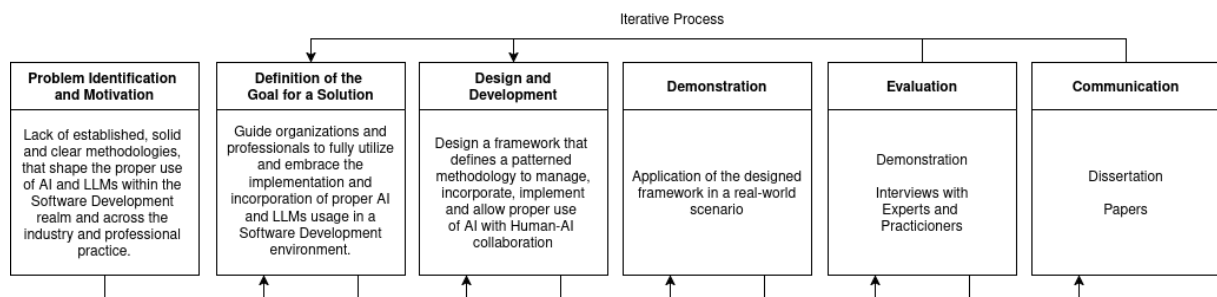


Figure 2.2: Phases of the Design Science Research Methodology

Problem identification and motivation addresses the lack of structured guidance, standardised methodologies and evaluation practices for integrating AI into software development while preserving quality, accountability and human oversight; the problem was identified through the *SLR*, which found fragmented practices and unclear human-AI role definitions across the *SDLC*. **Definition of objectives** derives from that problem objectives aimed at guiding organisations towards effective, secure and responsible adoption through appropriate processes and governance mechanisms. **Design and development** takes the review findings as input and produces the framework, defining phases, practices, hats and decision points, together with Apex as its reference implementation. **Demonstration** applies the artefact to a real project, exercising the phases end to end and producing documented artefacts, decision rationales and observations recorded against criteria fixed in advance. **Evaluation** separates

the two artefacts it assesses: the framework through interviews with practitioners and Agile experts and an analytical assessment against pre-stated criteria, the instantiation through a task-based study measuring perceived usability and cognitive workload with standardised instruments; that separation is argued in Chapter 9, since a usability result for the tool is not evidence that the method is sound. **Communication** disseminates results through a scientific article reporting the SLR, the extended abstract accompanying this dissertation, and the defence.

2.3 Mapping the Methodology onto this Document

Because DSRM prescribes activities rather than document sections, Table 2.1 states where each is carried out. The mapping is not strictly linear: as Chapter 7 describes, feedback gathered while building the reference implementation fed back into the framework's design, which is the iteration path DSRM explicitly allows.

Table 2.1: Mapping between DSRM activities and the chapters of this dissertation

DSRM Activity	Chapter	Output
Problem Identification and Motivation	Chapters 4 and 5	Evidence base from the SLR; formalised research problem
Definition of Objectives	Chapter 6	Objectives derived from the review findings
Design and Development	Chapters 6 and 7	The framework, and Apex as its reference implementation
Demonstration	Chapter 8	Application of the artefact in a real-world project
Evaluation	Chapter 9	Usability and workload measurement, interviews, analytical assessment
Communication	Chapter 10	Publications and dissertation defence

3

Research Background

Contents

3.1 Artificial Intelligence	6
3.2 Software Development Life Cycle	7
3.3 AI in Software Development	8
3.4 Vibe Coding	8
3.5 Spec-Driven Development	8

This chapter presents the theoretical background of the topics associated with the research. The topics covered are Artificial Intelligence, the Software Development Life Cycle, AI in Software Development, Vibe Coding, and Spec-Driven Development.

3.1 Artificial Intelligence

Artificial Intelligence (AI) refers to systems capable of performing tasks that traditionally require human cognitive abilities, such as reasoning, decision-making, pattern recognition, or language understanding. Early AI systems followed rule-based approaches, relying on explicitly defined logic to guide decision-making. More recent developments emphasize data-driven and statistical learning, where models improve their performance by identifying patterns in large datasets. AI now underpins a wide range of applications including recommendation systems, natural language processing, predictive analytics, and autonomous systems, reflecting its expanding relevance across different industries.

Generative Artificial Intelligence (GenAI) refers to a subset of AI systems designed to create new content, rather than classify or retrieve existing information. These systems generate text, images, audio, or code based on learned patterns in their training data. GenAI models operate by predicting likely future outputs given prior context, allowing them to produce coherent and contextually relevant content. This generative capacity has positioned GenAI as a significant innovation in domains that rely on iteration, prototyping, and creative exploration, including software development.

Large Language Models (LLMs) are a specific class of GenAI systems trained on vast corpora of text and source code. Based on transformer architectures, LLMs are capable of processing and generating sequences with contextual awareness. Their primary function is next-token prediction, but this capability enables more complex tasks such as summarization, explanation, reasoning, conversational interaction, and code generation. Within software engineering, LLMs are relevant because they can interpret programming intent expressed in natural language and generate syntactically valid code in a variety of programming languages. They are also able to explain, refactor, or restructure existing code, as well as produce tests, documentation, and other boilerplate artifacts with far less manual effort. These capabilities position LLMs as collaborative tools that can support both novice and experienced developers throughout different stages of software development. [1–5, 13–39]

3.2 Software Development Life Cycle

The Software Development Life Cycle (SDLC) describes the structured phases involved in building and maintaining software systems. Therefore, the SDLC is commonly structured into six main phases: requirements analysis, where system functionality is identified through requirements engineering; design, which defines the system architecture, data structures, and component interactions; implementation, focused on writing and integrating code; testing, where correctness, reliability, and performance are verified; deployment, which delivers the system to end users; and maintenance, involving ongoing updates, improvements, and operational support throughout the system's lifetime.

The SDLC is central to this research because LLMs can engage differently across its stages: they can support implementation by generating code, testing by producing test cases, and maintenance by summarizing or refactoring existing code.

Software development processes have evolved to emphasize iterative delivery and continuous integration, supported by methodologies such as Agile and DevOps. Despite these improvements, many development tasks remain manual, repetitive, and time-intensive. Developers spend significant time debugging, documenting code, maintaining consistency across repositories, and handling routine implementation tasks. As systems scale in complexity, these challenges become more pronounced, creating pressure to increase productivity without compromising maintainability or security. Organizations therefore seek tools that can automate or streamline specific tasks while maintaining developer oversight. This ongoing search for efficiency is a key factor driving interest in AI-assisted development workflows. [3, 4, 26, 35]

3.3 AI in Software Development

Advances in LLMs have introduced new forms of interaction between developers and software tooling. In what is often described as conversational or intent-driven programming, developers describe desired functionality in natural language and iteratively refine code generated by the model. This pattern represents a shift from writing code directly to supervising and guiding automated generation. Studies indicate that LLMs can accelerate prototyping, clarify unfamiliar codebases, and help maintain documentation quality. These capabilities can be especially valuable in early-stage development or when exploring new frameworks and libraries.

The integration of AI into the development process is increasingly characterized as collaborative. Rather than fully automating software creation, LLMs function as tools that support developers in tasks involving pattern recognition, summarization, or structural reasoning. The developer remains responsible for architectural decisions, validation, integration, and ensuring alignment with project requirements.

Research in this area focuses on how workflows adapt to that shift, how developers form trust in AI outputs, and how responsibilities are distributed between human judgment and automated generation. These themes are further examined in Chapter 4. [[3](#), [4](#), [13](#), [14](#), [22](#), [26](#), [28](#), [35](#)]

3.4 Vibe Coding

Vibe Coding refers to a style of software development in which developers describe their intentions, requirements, or constraints in natural language, and a generative model produces corresponding code or related artifacts. In this approach, the developer does not begin by writing syntactically precise code. Instead, development proceeds through a conversational interaction, where the model proposes implementations that are then reviewed and refined by the developer. While the model generates initial drafts or prototypes, the developer evaluates correctness, aligns the output with project requirements, and iteratively modifies the prompt or the produced code.

Vibe Coding shifts part of the programming effort from manual construction to specifying intent and assessing alternatives, which can influence how tasks are conceptualized and distributed during development. The approach does not replace traditional programming but reframes how initial structure and exploratory code are produced. As such, it has implications for developer workflows, cognitive load, and the distribution of effort across the SDLC. These implications, including reported advantages and limitations, are examined in detail in the Systematic Literature Review reported in Chapter 4. [[1](#), [2](#)]

3.5 Spec-Driven Development

Spec-driven development is an emerging engineering practice in which a persistent, versioned specification, rather than the source code itself, is treated as the primary artefact that both developers and AI agents work against. Instead of prompting a model directly against a codebase and accepting whatever it produces, the practice interposes a written specification, of requirements, design decisions, and constraints, that the model must consult and satisfy before code is generated, and that is kept in sync with the codebase as it evolves. Industry examples of this pattern include AWS Kiro's three-stage requirements, design, and tasks workflow, GitHub's Spec-Kit, which adds a "constitution" of immutable project

principles that generated code may not violate, and the Tessel Framework, in which, as its own materials put it, the specification itself becomes the maintained artefact rather than the code [40].

The practice is tracked by Thoughtworks' Technology Radar as an emerging AI-assisted engineering technique, named specifically "spec-driven development" [40], and has begun to receive academic attention: Farrag frames it as specification-driven governance, arguing that it addresses what is termed the productivity-reliability paradox in AI-augmented development, though this framing is currently reported only in a preprint that has not yet completed peer review [41]. The practice responds directly to the risk profile characterised in Chapter 1: narrowing and anchoring the context a model is permitted to act upon is a governance mechanism, not merely a convenience, since it gives generated output a stable reference to be validated against rather than leaving correctness to be inferred from the prompt alone. This precedent is the direct grounding for Spec-Anchored Continuity, the persistent specification mechanism at the centre of the framework proposed in Chapter 6.

4

Systematic Literature Review

Contents

4.1 Planning the Review	10
4.2 Conducting the Review	12
4.3 Reporting the Review	13
4.4 Discussion	21

This chapter describes the execution of the systematic literature review, following the three SLR stages: Planning, Conducting, and Reporting the results.

4.1 Planning the Review

The planning stage establishes the motivation for the review, the research questions the search addresses, and the search method employed.

4.1.1 Motivation

The integration of AI, particularly LLMs and GenAI, represents a significant transformation in Software Engineering. These technologies are increasingly adopted across the SDLC to support code generation and completion, documentation, requirements engineering, automated bug fixing, and developer learning, and their use has been associated with productivity gains, improved efficiency, and support for complex development tasks [3, 4, 13, 14, 22, 26, 28, 35].

Adoption has nevertheless outpaced the resolution of the challenges it raises, which is what motivates systematic investigation. The literature reports persistent concerns over the quality, reliability, and trustworthiness of AI-generated artifacts, and methodological barriers such as limited explainability and the black-box nature of advanced models. Effective use of LLMs also remains highly dependent on prompt formulation, while existing research concentrates on isolated tasks, leaving several SDLC phases, particularly requirements engineering and design, with limited practical guidance.

This review therefore maps and synthesises existing research on AI and LLM use in software development, organising the evidence on benefits, limitations, methodologies, stakeholder perspectives, and application domains so that research gaps become identifiable.

4.1.2 Research Questions

This overly general topic admits many possible research questions; the five below are the ones judged specific and relevant enough to answer with a systematic review:

Table 4.1: Research Questions and Focus Areas

RQ	Research Question	Focus
RQ1	What are the benefits of using and incorporating Vibe Coding, AI, and Large Language Models in software development?	Identify and categorize improvements in productivity, creativity, code quality, automation, collaboration, and innovation enabled by AI-driven tools.
RQ2	What are the consequences that may arise from using and incorporating Vibe Coding, AI, and LLMs in software development, both in code generation and other sections?	Analyze risks such as technical debt, bias, reduced developer learning, code ownership, ethical issues, and dependency on AI tools.
RQ3	What methodologies and management methods are being used to integrate Vibe Coding, AI, and LLMs into software engineering processes and workflows?	Explore frameworks, best practices, and process models (e.g., AI-augmented Agile, human-in-the-loop development, prompt engineering practices).
RQ4	What are developers' and other stakeholders' opinions regarding the use of AI and LLMs in software development?	Assess perceptions, trust, satisfaction, and acceptance of AI-assisted coding among developers and organizations.
RQ5	Which industry sectors are adopting AI for application development, and how do they benefit from its integration?	Map adoption trends across industries (e.g., finance, healthcare, IT, education), highlighting sector-specific motivations and outcomes.

4.1.3 Review Protocol

The review begins with a literature search: a search string is defined and executed against the chosen data source. The criteria used here, stated in Table 4.2, returned 646 articles.

Table 4.2: Research Elements and Details

Element	Research Details
Source	EBSCO
Final Search String	("Vibe Cod*" OR "AI models" OR "AI agen*" OR "large language model*" OR "generative AI" OR openai OR "chatgpt" OR LLM* OR "claude" OR "gemini" OR copilot OR co-pilot) AND ("software development" OR "application development" OR "code generation")
Search Method	Search string found in the abstract of articles in English academic journals or conference materials, with no date range limit
Results	646

The subject of this review, the use of AI and LLMs to develop applications, can be expressed in many different ways, and the string had to absorb that variation. "Vibe Coding" is the most recent term for the practice, but the practice predates the term: querying it alone returned few relevant articles, because the same work is published under different expressions. Synonymous keywords were therefore added, and it is the combination of "code generation" and "application development" with "generative AI" and "AI models" that retrieves the intended body of work rather than any single term.

The string was executed against the EBSCO Online Digital Library through its Advanced Search option, over the AB abstract field and the title, restricted to full-text-available results.

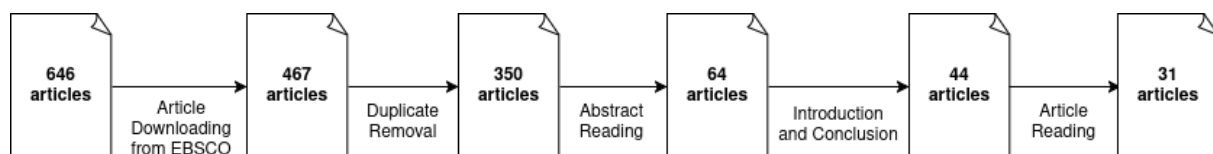
4.2 Conducting the Review

The second stage applies the filtering criteria to the 646 articles retrieved in the first. The criteria are stated below and the attrition at each step is shown in Figure 4.1.

4.2.1 Filtering Process

Extracting all 646 articles was not possible: EBSCO's page allows the download of 50 articles at a time, and after reloading and extending the page to its maximum the site could display, in relevance order, only 467 of the 646.

The AI-powered systematic review management platform "Rayyan" was used to organise the articles. Of the 467 extracted, 227 duplicates were identified and removed, leaving 350 for abstract reading; reading those abstracts to establish the topic covered rejected 286 as belonging to a theme other than developing applications using AI, leaving 64 articles.

**Figure 4.1:** Articles Filtering Process

The content of those 64 articles was then read and each was placed in one of four categories: Included, Maybe, Excluded and Skipped. Articles were "Skipped" when reading the introduction and conclusion showed them to be off-theme, insufficiently recent, which matters given how fast the field moves, or silent on all five research questions; 20 were skipped and excluded on those grounds. "Maybe" was used while it was unclear whether an article's content bore on the theme; most were ultimately excluded in favour of more recent studies reflecting the current state of the art, and those that did fit the topic were included.

An article that answered any of the five research questions clearly and explicitly was included. Where the information was unclear, insufficient, or too thin to confirm that an answer was given, the predefined inclusion and exclusion criteria in Table 4.3 decided eligibility. Of the 64 articles, 31 met the criteria and were retained for analysis.

Table 4.3: Inclusion and Exclusion Criteria

Inclusion Criteria	Exclusion Criteria
• Relevance regarding AI and LLM usage in the development of software applications • Relevance regarding benefits and consequences of incorporating and using AI and LLMs in the development of software applications • Relevance regarding methodologies and management methods used to integrate AI and LLMs into software engineering processes and workflows • Relevance regarding people's opinions with AI and LLMs use in software development • Relevance regarding which industry sectors are adopting AI for application development and its benefits	• Not written in English • Does not clarify any of the Inclusion Criteria • Outdated due to the highly dynamic nature of the field (anything older than 2024)

4.2.2 Data Extraction Analysis

The retained articles were then subdivided by the venue and year in which they were published. None of them is categorised as conference material: every retained article is a peer-reviewed academic journal article as classified by EBSCO, distributed over time as shown in Figure 4.2.

4.3 Reporting the Review

The third and last stage reports what the analysis of the retained articles established, answering each research question in turn. The distribution of the retained corpus by publication venue is given in Appendix A.1; it is concentrated rather than diffuse, with ACM Transactions on Software Engineering and

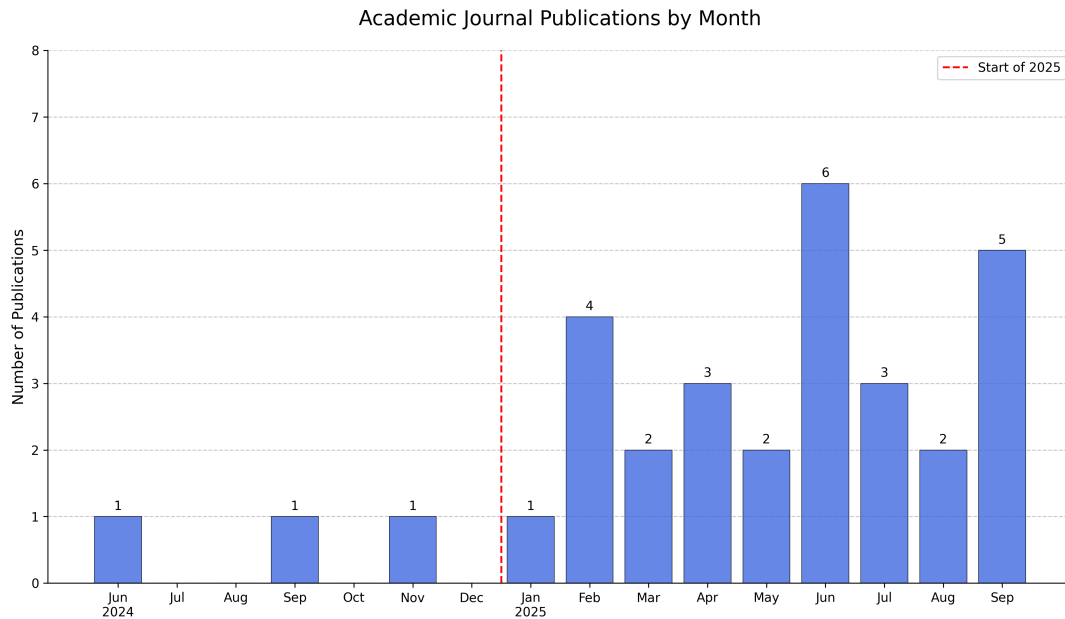


Figure 4.2: Academic Journal Publications

Methodology contributing ten of the retained articles and Information and Software Technology three, while the remaining venues contribute one each, which is consistent with a topic that is being actively published across the field rather than owned by any single outlet.

4.3.1 Benefits of AI and LLMs usage in Software Development

The first research question concerns the benefits of AI and LLM adoption for developers, companies and other stakeholders. Table 4.4 collects the benefits reported in the literature; what follows analyses them.

Table 4.4: Benefits of using AI and LLMs in Software Development

Benefits	Description	#	References
Improved Code Quality, Accuracy, and Traceability	"In practice, it has been proven that using Generative AI can increase efficiency and productivity by improving code quality and optimizing work with faster times." [14].	20	[3, 4, 13–16, 18, 19, 21–24, 26, 27, 29–31, 33, 34, 36]
Increased Productivity and Efficiency	"Accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules, dramatically shortening development cycles" [1].	18	[1–4, 13, 14, 19, 20, 24, 27, 29, 30, 33–37, 39]

Continued on next page

Benefits	Description	#	References
Automation of Testing, Debugging, and Repair	AI and LLMs "can increase the efficiency of software development by automating coding, bug detection, debugging, and personalizing software according to user needs" [14].	12	[1, 3, 16, 17, 20, 21, 29, 30, 33–35, 37]
Optimization of Code Performance and Efficiency	"The proportionally small number of studies in SE subfields like Build, Release, Deploy, Operate and Monitor indicates a broad untapped area for cost savings and novelty, which could bring major optimisation in Software Engineering" [28].	10	[1, 2, 4, 13, 14, 17, 24, 28, 33, 35]
Support for Early Software Development Phases (Requirements & Design)	AI and LLMs have effectiveness in "...enhancing stakeholder communication and requirement documentation" while noting challenges with bias and accuracy in requirements and design elicitation [29].	8	[4, 15, 16, 22, 27, 29, 33, 35]
Automated Documentation and Summarization	"LLMs can [...] automatically generate documentation for codebases, including explaining the purpose and functionality of specific functions, classes, and modules. This ensures that documentation stays current and can be updated as code evolves, providing developers with easily understandable, well-structured explanations of how various parts of the system work." [24].	8	[4, 13, 17, 22, 24, 26, 35, 36]
Enhanced Security and Vulnerability Management	Gen AI technology good practices enhances SDLC security by supporting development through "vulnerability detection, threat modeling, secure coding practices, and incident response" [17].	6	[17, 18, 26, 33, 35, 39]
Knowledge Acquisition and Understanding	"Reduce technical barriers and transform the programming process into a collaborative, conversational, and more accessible act" [2].	5	[2, 13, 20, 27, 33]
Management and Organizational Support	Assist in managing and "curating organizational information about day-to-day organizational practices" [27].	5	[3, 4, 16, 17, 22]
Developer Well-being and Satisfaction	AI tools improve developer satisfaction by enabling developers to "prioritize interesting and fun tasks through automating menial, boring tasks" [27].	4	[4, 19, 27, 33]
Democratization and Accessibility	AI integration aims to "shift the programming paradigm from a traditional code-centric approach to one that involves verbal interactions between the programmer and the artificial intelligence agent being used", making programming more accessible [2].	2	[1, 2]

The two most frequently reported benefits, productivity and code quality, are also mutually reinforcing: acceleration does not come at the cost of quality, since AI helps generate code adhering to high standards and thereby raises the accuracy and particularly the traceability of the final product [30]. Automating the time-consuming quality-assurance tasks, testing, debugging and repair, supports the same synergy, and the mechanism is specific: LLMs leverage explicit feedback such as compiler error mes-

sages in follow-up prompts, iteratively fixing their own coding errors within a single conversation context.

Beyond the direct impact on the codebase, AI also improves developer job satisfaction and democratises programming knowledge, by automating monotonous tasks so developers can prioritise more engaging work, and by lowering technical barriers so that programming becomes a more conversational, accessible activity for a broader audience.

4.3.2 Consequences of AI and LLMs usage in Software Development

The second research question concerns the consequences that may arise from incorporating AI and LLMs into software development, in code generation and elsewhere. Those reported across the corpus are collected in Table 4.5.

Table 4.5: Consequences of using AI and LLMs in Software Development

Consequences	Description	#	References
Increased Operational Costs and Resource Constraints	"The financial burden can become substantial and must be considered when LLMs are deployed to handle a large number of tasks" [5].	20	[3, 5, 20, 22–24, 26, 28, 35, 38]
Reliability, Accuracy, and Hallucination Risks	This is particularly problematic in complex or niche scenarios. "Because this AI can simply make things up or sometimes spit out things that don't even exist" [13].	15	[2, 4, 15, 16, 18–20, 22, 23, 30, 31, 34–37]
Security Vulnerabilities in Generated Code	"Recent studies suggest that code generated by AI models often contains security vulnerabilities that may follow patterns different from those in human-written code. Evidence suggests that AI-generated code may harbor unique security risks in areas such as cryptographic implementation, memory management, and error handling" [39].	15	[3, 13, 14, 16–20, 24, 26, 28, 33, 36, 37, 39]
Ethical, Legal, and Intellectual Property Issues	"Questions about the ownership and licensing of that code will arise, particularly when the models are trained on publicly available, open-source projects" [24].	8	[2, 3, 13, 17, 19, 22, 24, 35]
Homogeneity and Bias Propagation	As LLMs increasingly train on LLM-generated code, there is a risk of code homogeneity, reducing diversity in coding practices and potentially perpetuating slow, biased, or insecure patterns. This reduces the diversity of code practices over time, as LLMs tend to "reaffirm patterns seen in their training data (e.g., public GitHub repositories)" [35].	8	[3, 13, 14, 17, 19, 24, 34, 35]
Evaluation and Standardization Challenges	"It is urgent to establish an effective, objective, and complete evaluation system for large language models for code" [3].	7	[3, 14, 15, 23, 26, 32, 39]

Continued on next page

Consequences	Description	#	References
Risk of Over-reliance and Cognitive Decline	"Of course, sometimes sometimes there is almost the danger that you stop thinking for yourself a little. I think that might also be a problem in the future, that if AI is constantly available, people will tend to become lazy and stop thinking for themselves" [13].	4	[2, 13, 19, 27]
New Overhead and Workflow Inefficiencies (Human Review)	Integrating AI requires developers to shift their focus, often spending more time reviewing, verifying, and refining outputs, rather than writing new code because LLMs still "require human oversight and review of their outputs before implementation" [29].	4	[2, 13, 22, 29]
Ineffectiveness of Existing Security Tools	Existing tools detect only "46.7% of vulnerabilities in AI-generated code, a significantly lower rate than for human-written code vulnerabilities (66.4%)" [39].	3	[17, 26, 39]

The consequences tabulated above are technical, ethical and operational, and the operational and technical ones are cited most often. Cost comes first: training, fine-tuning and deploying large-scale models can be substantially more expensive than traditional methods, offsetting the productivity gains they are meant to buy. Reliability comes second: LLMs are fallible and frequently produce plausible but fabricated output, "hallucinations", most problematically in complex or niche scenarios. The detection gap the table records between AI-generated and human-written code is the same problem seen from the tooling side, existing instruments being matched to failure modes AI-generated code does not share [39].

Beyond the technical domain, adoption carries risks to human factors, ethics, and economic viability. Over-reliance risks eroding developers' independence and problem-solving skills as instant assistance becomes the default. Bias embedded in training data risks being perpetuated and magnified in generated code, particularly in sensitive industries [24], and code ownership and licensing remain unresolved where models train on open-source code [35].

Together these risks point to the same conclusion: LLMs' non-deterministic output and the absence of standardised evaluation benchmarks make extensive human review non-optional before AI-generated artefacts reach production, and the time that review costs is itself part of the true cost of adoption, not a footnote to it.

4.3.3 Methodologies and management methods used to integrate AI and LLMs in Software Development

The third research question asks which methodologies and management methods are used to integrate AI and LLMs into software development. Table 4.6 lists those the corpus reports.

Table 4.6: Methodologies and Management Methods used to integrate AI and LLMs in Software Development

Methodologies	Description	#	References
In-Context Learning (ICL) / Few-shot Learning	"ICL seeks to improve the abilities of LLMs by integrating context-specific information, in the form of an input prompt or instruction template, during the inference and thus without the need to perform gradient-based training" [38].	9	[15, 16, 20, 22, 26, 33–35, 38]
Prompt Engineering (PE) / Prompt-Driven Development	"The effectiveness of prompt engineering critically determines the performance of leveraging the capabilities of LLMs". [26]	5	[3, 20, 26, 32, 33]
Chain-of-Thought (CoT) Prompting	"A CoT is several intermediate natural language reasoning steps that lead to the final answer... the CoT only uses natural language to describe how to solve a problem step by step sequentially". [31]	5	[3, 16, 20, 26, 31]
Human-In-The-Loop (HITL) Paradigm	"While overall, LLMs exhibit decent reliability in their outputs with varying degrees, they still require human oversight and review of their outputs before implementation". "The importance of the human contribution in identifying and resolving problems to maintain software quality is widely acknowledged by many authors, which also gives rise to the Human-in-the-Loop (HITL) paradigm" [29].	5	[15, 24, 29, 35, 37]
Structured Prompting / Structured Chain-of-Thought (SCoT)	"SCoT denotes several intermediate reasoning steps constructed by programming structures", and "SCoT prompting asks LLMs first to generate an SCoT and then output the final code." [31]	4	[23, 26, 30, 31]
Multi-Agent Systems / Self-Collaboration Approach	"Different agents play different roles and collaborate to fulfill user requirements". "[...] a self-collaboration framework with role instruction, which allows LLM agents to collaborate with each other to generate code for complex requirements" [21, 37].	4	[13, 18, 21, 37]
Integration with External Tools and Domain Knowledge	"Combining the linguistic capability of LLMs with specialized domain knowledge to enhance output quality, functionality, or security". "The integration of relevant context into prompts may further improve current GenAI models, thereby reducing hallucinations and increasing reliability". [13, 37]	3	[3, 4, 13, 37]
AI-Assisted Pair Programming / Human Oversight	"LLMs still require human oversight and review of their outputs before implementations, and they present unique characteristics to be considered before adoption". "The AI orchestrator will coordinate the perception and action of each agent" [29, 35].	3	[2, 29, 35]
Retrieval-Augmented Generation (RAG)	"RAG have gained attention. ICL and RAG offer a low-computation option by eliminating the need for parameter updates". "RAG demonstrates higher effectiveness than ICL but falls short of LoRA's effectiveness across all three CodeLlama model variants". [38]	2	[20, 38]

Continued on next page

Methodologies	Description	#	References
AI-Augmented Agile Development (e.g., AgileGen Framework)	"[...] we propose a human-agent collaborative generative software development approach, focusing on human involvement at the beginning (scenario decision-making) and end (acceptance and recommendation decision-making) of each iteration" [4].	1	[4]
Utilizing Gherkin Language (Behavior-Driven Testing)	"[...] we use Gherkin to supplement unclear user requirements with well-defined acceptance criteria, guiding the generation of applications that align with user requirements" [4].	1	[4]
Vibe Coding	"[...] emerging concept of vibe coding, in which artificial intelligence (AI) accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules" [1]	1	[1]
Integration of LLMs with Traditional Testing	"the integration of LLMs with traditional approaches has emerged as an important direction for future exploration". "By combining them together, there can be 1.4× to 2.3× improvement than the baselines". [37]	1	[37]

The methods reported range from prompt-level tuning to large-scale process frameworks, and they stratify. The fundamental layer improves model interaction through prompts, led by In-Context Learning (9 articles, the most frequently discussed technique) and Prompt Engineering, whose quality critically determines LLM effectiveness; the same principle motivates Chain-of-Thought prompting, in which the model generates intermediate reasoning steps toward its answer, and its Structured variant, which introduces programming constructs such as sequence, branch and loop into those steps [31].

Given LLMs' variable reliability, the Human-In-The-Loop paradigm is considered mandatory for maintaining software quality [29]: developers retain critical oversight, reviewing and verifying AI output before it is implemented. This is formalised in frameworks such as AI-Augmented Agile Development (AgileGen), which structures collaboration into lightweight iterations and reserves human decision-making for the start (scenario definition) and end (acceptance) of each cycle [4], preventing the error accumulation that unsupervised multi-agent systems risk.

"Vibe Coding", mentioned once in the corpus, represents a conceptual leap in prompt application: translating abstract intent directly into functional code, particularly for domain-specific tasks such as biomedical software [1]. It is recent and not yet widely studied, but reliably translating a "vibe" into working software is exactly what the methodologies above exist to constrain.

4.3.4 Developers' and other stakeholders' opinions regarding the use of AI and LLMs in Software Development

The fourth research question concerns the human perspective, and the reception of these tools proved to differ enough between groups that the extracted opinions are tabulated separately by group, in Appendix A.2: the public and researchers, developers and practitioners, and organisations and manage-

ment. The separation is itself a finding, because the three groups do not agree.

Among the public and researchers, the dominant position is scepticism grounded in fallibility rather than in capability. General perception, cited in six articles, treats these systems as a mixture of software and sorcery, and the primary technical barrier reported is misinformation risk: generated code may execute without error while failing to meet the intended requirement. That risk is amplified by variability, since the same prompt repeated yields different code each time. This is what makes the single most prevalent opinion across all three groups, the need for human oversight with seven articles, follow from the evidence rather than from caution: errors such as inconsistencies in design diagrams or incomplete implementations propagate into project integrity.

Developers and practitioners report the opposite emphasis. Productivity and efficiency gains are the most-cited positive opinion for this group, in six articles, and LLMs are valued for knowledge acquisition because they allow a developer to bypass extended search or dependence on colleagues [13]. Two countervailing findings appear alongside it. The convenience carries a documented risk of over-reliance and erosion of technical competence, and effectiveness is not seamless: prompt design is reported as a significant time investment requiring considerable skill, which reduces the efficiency it promises [13].

Organisations and management treat adoption as a risk-management exercise. The strategic case is tied to job satisfaction, talent retention and competitiveness, since automating routine work retains valuable staff [27], but it is constrained by the group's most-cited concern, data security and privacy, in six articles. The consensus strategy is to avoid commercial LLMs in favour of open models fine-tuned on organisation-specific data, which in turn raises operational cost and pushes teams towards medium-sized models to manage computational limits. Expectation management appears less often but matters for the same reason: organisations report needing to discount the hype surrounding these tools when planning their integration.

4.3.5 Industry sectors adopting AI and LLMs for Software Applications Development

The fifth research question asks which industry sectors are actively adopting AI for application development and what sector-level benefits they report. The extracted evidence is tabulated in Appendix A.3; what follows is what it establishes.

Adoption concentrates overwhelmingly in Software Engineering, Software Development and IT Consulting, which is unsurprising since these domains are both the origin and the primary proving ground for LLM-based tools. The reported use cases centre on code generation, completion, explanation and refactoring, and several sources describe a gradual shift in the developer role from writing code manually to orchestrating AI-enabled workflows in which the human directs, validates and refines automated output [4, 27, 29]. In consulting specifically, LLMs are valued for accelerating onboarding and transferring client-specific business knowledge between projects. Adoption also appears in specialised technical domains involving infrastructure-level or mathematically intensive systems, where models must be adapted or augmented with structured knowledge to produce reliable output [3, 5, 17, 37].

Beyond the core **IT! (IT!)** sector, adoption clusters in high-stakes regulated environments: Finance, Banking and Regulated Industries in eight articles, Transportation and other safety-critical domains in seven, Healthcare and Biomedical Research in five, and Legal Reasoning in four. In these domains

LLMs are introduced cautiously, with emphasis on reliability, auditability and regulatory compliance, and the literature is consistent that extensive human review and verification remain mandatory wherever incorrect or unverifiable output could produce safety risks, compliance failures or legal liability [1, 3, 26, 33]. Education and Training, in five articles, is an emerging area in which LLMs act as adaptive learning companions rather than replacements for educators, supporting exercise generation, personalised feedback and formative assessment [2, 15, 34].

No sector reports adopting these tools without a human check, and the sectors with the most to lose report the most explicit checks. That is a finding about process rather than about capability, and it is the point at which this review begins to constrain the design of a framework rather than merely describing practice.

4.4 Discussion

Read across the five research questions rather than one at a time, the review converges on a single pattern rather than five separate ones. Every benefit reported under RQ1 has a corresponding risk reported under RQ2 that threatens the same property it claims: speed and code quality (RQ1) sit against hallucination, security vulnerabilities and homogenisation (RQ2); democratised access sits against over-reliance and skill erosion. RQ3 shows the field's answer to that tension is not automation but control, since every prevalent method, In-Context Learning, Prompt Engineering, Chain-of-Thought and above all Human-In-The-Loop, exists to shape or supervise AI output before it is trusted, and no method proposes removing the human. RQ4 shows the same tension held by the people who do the work: every stakeholder group pairs enthusiasm with a caveat, and human oversight is the one point of consensus that survives across public, practitioner and organisational perspectives despite their differing incentives. RQ5 shows it is also the organising variable for adoption itself: where an incorrect output is cheap, adoption is broad and unconditional; where it is expensive, as in finance, transportation or healthcare, adoption is conditioned explicitly on verification. The absence of a standardised, secure integration methodology, not a shortfall in model capability, is therefore what the literature identifies as the actual barrier to broader adoption, and it is the gap this dissertation's proposal is built to close.

Table 4.7 restates this pattern at the granularity a framework needs: the specific contribution and the specific challenge AI introduces at each phase of the SDLC, from requirements through maintenance.

Table 4.7: Impact of AI and LLMs across Software Development Lifecycle Phases

SDLC Phase	Main AI Contributions	Main Challenges
Requirements Analysis	Support for requirements elicitation, clarification, summarization, and stakeholder communication; rapid exploration of solution ideas through conversational interfaces.	Risk of ambiguity amplification, overconfidence in AI-generated requirements, and lack of domain context; increased need for human validation early in the process.

Continued on next page

SDLC Phase	Main AI Contributions	Main Challenges
Design	Generation of architectural alternatives, design patterns, API suggestions, and exploratory prototypes.	Limited understanding of system-wide constraints; risk of inconsistent or suboptimal architectural decisions without expert oversight.
Implementation	Code generation, refactoring, boilerplate reduction, and rapid prototyping; strong association with practices such as Vibe Coding.	Hallucinated code, inconsistent outputs, security vulnerabilities, and reduced trust due to output variability; heavy reliance on human review.
Testing	Automated test case generation, test data creation, bug localization, and support for exploratory testing.	Difficulty validating correctness of AI-generated tests; increased testing complexity due to unpredictable AI-generated code.
Deployment	Support for configuration scripts, infrastructure-as-code templates, and documentation generation.	Risk of misconfiguration or insecure defaults; limited contextual awareness of operational environments.
Maintenance	Assistance with debugging, documentation updates, code comprehension, and technical debt identification.	Potential over-reliance on AI explanations; challenges in maintaining long-term system understanding and accountability.

Read down its challenge column, the same requirement recurs at every phase: a human check of the model's output against something written before that output existed.

5

Research Problem

This chapter presents the first activity of the DSRM, the identification of the research problem and its motivation. The second activity, the definition of objectives, is presented together with the design of the artefact that satisfies them in Chapter 6, since the objectives derived here are what that design is answerable to.

The review reported in Chapter 4 shows that AI and LLMs are being adopted across sectors and across every phase of the SDLC, and that the field is evolving fast enough for new models and tools to appear continuously. It also shows that the benefits are accompanied by challenges that obstruct successful implementation. The most consequential of these, and the one reported most consistently, is the absence of established, clear methodologies shaping the proper use of these tools in professional software development [13–16].

Several frameworks and approaches that incorporate AI into the development process have been proposed, and are summarised in Table 4.6. They differ significantly in scope, assumptions and level of abstraction, and share no common foundation regarding lifecycle integration, management practice or evaluation criteria [3]. Organisations therefore face difficulty not only in adopting AI-supported practice but in assessing its impact on software quality, security and maintainability, because no standardised benchmarks or performance metrics exist against which to do so. The field is actively searching for structured and repeatable integration patterns rather than converging on them.

Established methodologies do not supply those patterns. Waterfall, Scrum, Kanban and the Agile Manifesto introduced substantial improvements in adaptability and collaboration, but were designed for environments in which every contributor is human [4]. In AI-driven contexts they fall short in three specific ways. High-speed AI-assisted coding accelerates implementation while creating pressure upstream, in requirements clarification, and downstream, in testing, verification and maintenance. They provide no explicit structure for governing AI participation, for assessing model reliability, or for mitigating the

resulting risks. And they assume all contributors are human, whereas a model is a new category of collaborator whose output carries no author and therefore no obvious owner.

The empirical case for treating this as a process problem rather than a capability problem is set out in Chapter 1 and is not restated here. Two independently reported findings carry it. Security performance does not improve as functional correctness improves, so generated code that works is not thereby safe [9]; and acceleration at the point of authorship relocates to human review rather than disappearing, without reaching delivery outcomes [12]. Both sources are industry-reported rather than peer-reviewed and are cited as such. Their value lies in agreeing on the mechanism rather than in the precision of any single figure: neither security nor delivery improves as a by-product of faster generation, and both depend on where validation and review sit in the process.

The core research problem addressed in this work is therefore the lack of patterned guidance, methodological rigour and standardised evaluation metrics for the systematic integration and management of AI and LLM-based tools across the SDLC, while preserving human oversight, software quality and security, and organisational accountability. This gap limits the ability of organisations to realise the benefits of AI-assisted software development sustainably.

The objectives a solution must satisfy, and the design that answers them, are presented in Chapter 6.

6

Research Proposal

Contents

6.1 Objectives	26
6.2 Purpose and Scope	26
6.3 Design Principles	27
6.4 Lifecycle Phases and the AI Interaction Mapping	28
6.5 Roles and Responsibilities	30
6.6 Governance Mechanisms and Quality Gates	32
6.7 Work Organisation	35
6.8 Operational Playbooks	35
6.9 Positioning Against Alternatives	37
6.10 Summary	38

This chapter presents the second and third activities of the DSRM: the definition of objectives for a solution to the research problem formalised in Chapter 5, and the design of the artefact that satisfies them, a process-oriented framework for governed Human-AI collaboration across the SDLC. The framework is described here as a methodology, independently of any implementation; its instantiation as a working system is the subject of Chapter 7.

6.1 Objectives

The systematic literature review in Chapter 4 established the current state of AI and LLM adoption in software development, and the analysis of its benefits and consequences led to the problem formalised in Chapter 5: the lack of structured guidance, methodological rigour and standardised evaluation practice for integrating these tools into development processes. The research objectives are formulated to address that problem directly, and Table 6.1 links each identified gap to the framework objective that resolves it.

Table 6.1: Mapping between Research Questions, Key Findings, Problem Aspects, and Framework Objectives

RQ	Key Finding from SLR	Problem Aspect Addressed	Resulting Framework Objective
RQ1	Usage is unevenly distributed across SDLC phases and lacks structured integration.	No structured guidance on where and how AI should be applied across the SDLC.	Define a process framework mapping AI-supported activities to specific phases.
RQ2	Recurring risks in reliability, validation, security and insufficient human oversight.	Insufficient methodological support for risk, quality and accountability.	Integrate governance mechanisms, human-in-the-loop practice and quality gates.
RQ3	Significant variation among approaches and no consensus on managing adoption.	Fragmentation of existing AI adoption methodologies.	Consolidate practice into a unified, repeatable process-oriented framework.

The remainder of this chapter presents the framework designed to meet these three objectives.

6.2 Purpose and Scope

The artefact is a method in the sense the DSRM gives that term: phases, hats, artefacts and decision rules prescribing how a team organises the participation of AI across the SDLC. It is stated at the level of process rather than of tooling, so that it can be assessed independently of the system instantiating it in Chapter 7 and adopted against a different tool stack. It covers AI-assisted activity in every phase rather than the code-authoring phases alone, the collaboration and model-selection practices that activity requires, expressed as process obligations rather than a catalogue of prompts, and the governance mechanisms of Section 6.6.

It excludes the training or fine-tuning of LLMs, model architecture design, Machine Learning Operations (MLOps) pipelines and tool-specific detail. These are boundaries rather than omissions, following from Chapter 5's concern with teams that *consume* general-purpose models rather than *produce* them: MLOps governs a model's lifecycle as a deployed asset, while this framework governs the lifecycle of software a model helped write, and a pipeline can be entirely mature under the first while the second remains ungoverned, which is the deficiency Chapter 4 reports. Tool-specific detail is excluded for the same reason Chapter 7 and Chapter 9 are kept separate: a framework prescribing a vendor cannot be evaluated independently of it.

6.3 Design Principles

Eight principles govern every subsequent design decision. Each is stated in Table 6.2 together with the evidence that motivates it, since a principle asserted without grounding is a preference, and a framework built out of preferences cannot be defended as a research contribution. Where the grounding is a design commitment rather than an empirical finding, that is said so.

Table 6.2: The eight design principles and the evidence grounding each

Principle	Statement and grounding
Human-in-the-Loop by Design	Every AI-generated artefact is subject to explicit human review and acceptance before entering the product. Adoption and trust diverge, at around 90% daily use against roughly 30% reporting low or no trust [7], and benchmarking shows the distrust warranted: the best agent and model combination was functionally correct 61% of the time but secure only 10.5% [8], and 45% of samples across more than a hundred models introduced an OWASP Top 10 vulnerability, with security flat as correctness improved [9].
AI as Augmentation, Not Substitution	Humans retain responsibility for technical, architectural and quality decisions. In a randomised controlled trial developers took 19% longer with assistance while estimating they had been 20% faster [11]; substituting judgement removes the only faculty able to notice such a discrepancy, and the SLR reports reduced code understanding as a recurring consequence of unsupervised delegation [3, 4].
Process-Level Governance of AI Usage	AI participation is governed through repeatable mechanisms making contributions visible, traceable and attributable. AI amplifies: capable teams become measurably more capable, dysfunctional teams measurably less so [7]. If the surrounding process sets the sign of the effect, governance must live in the process rather than in prompt discipline.
SDLC-Wide and Phase-Aware Integration	Participation is defined per phase, with hats and controls adapted to that phase's risks. Usage is unevenly distributed and concentrated on code generation, leaving requirements, design and maintenance with little guidance [3–5]; governing only the well-served phases would reproduce the gap the framework exists to close.
Hats, Not People	Every role named is a function that must be performed, not a person who must exist. A design commitment rather than an empirical claim, argued in Section 6.5; its visible consequence is that no role is named whose only content is oversight.
Explicit Responsibility and Accountability	For each AI-assisted activity a named hat is accountable for initiation, validation and acceptance. When an artefact has no identifiable author no party is answerable for its defects, and ownership is among the concerns most frequently raised by practitioners and organisations [3, 13]. Accountability attaches to a function so that it survives the absence of any individual.
Risk-Proportional Validation and Control	Validation rigour is proportional to potential technical, organisational and ethical risk. The defect profile is not uniform: vulnerability rates reach 72% for Java against 45% overall and cluster in specific weakness categories [8, 9]. A uniform control is either too costly for the routine case or too weak for the dangerous one, which is why remediation separates a Fast Lane from a Secure Lane.
Spec-Anchored Continuity	A persistent versioned specification is maintained alongside the codebase and is what every AI-assisted activity is constrained by and validated against. As established in Section 3.5 this instantiates spec-driven development rather than inventing it [40, 41]. The limitation belongs with the claim: its own principal source describes the practice as emerging and its scaling behaviour as an open risk [40], which Chapter 9 treats as something to test rather than assume.

6.4 Lifecycle Phases and the AI Interaction Mapping

This section answers DRQ1. The framework organises the lifecycle into six phases, preceded by a Strategic Alignment stage that sits outside the lifecycle proper because it is performed before the Squad is engaged at all: it decides whether a Unit of Work should exist, whereas the six phases decide how one is built. The six are Discovery and Requirements; Design, Prototyping and Architecture; Implementation; Testing; Deployment; and Maintenance. Table 6.3 fixes, for the stage and each of them, what enters, what the model is permitted to produce, what leaves, which hat is accountable, and which control must be satisfied before exit. It is the central artefact of this chapter and is not restated in prose here; what follows argues the two points on which the model departs from its closest published precedent, and then why the order of the phases is not the shape of the process.

Two mechanisms the table names are defined here, since both recur throughout the chapter. The **Interactive Bridge** is the interaction pattern governing Discovery and Requirements: the model drafts scenarios in plain natural language for human critique, the Trio confirms, adds, deletes or modifies them, and only the approved result is compiled into formal Gherkin and anchored. Separating critique from compilation matters because natural language is the medium in which a domain expert can detect a wrong assumption, whereas formal syntax invites review of the syntax instead. The **Consistency Factor** is the strict automated test set produced before generation begins, derived from both the functional and the technical specification, and it constrains what the model may produce rather than assessing it afterwards; it is specified in full in Section 6.6.1.

The framework's most direct precedent is the AI-Driven Development Lifecycle (AI-DLC) published by Amazon Web Services [42], which organises work into Inception, Construction and Operations, replaces sprints with *Bolts*, short cycles measured in hours or days, replaces Epics with *Units of Work*, and governs AI participation through collaborative rituals of which *Mob Elaboration* is adopted here; these three terms are taken directly from AI-DLC with no originality claimed for them, and its evidential status is stated with equal directness: its reported ten-to-fifteen-fold productivity improvement is vendor-reported, unreplicated and uncarried here as evidence, adopted only as process vocabulary. Most of the expansion from three phases to six is decomposition rather than disagreement, since Discovery and Requirements together with Design, Prototyping and Architecture occupy the territory it calls Inception, and Implementation corresponds to Construction. Two extensions are substantive.

The first is a decoupled Testing phase. In AI-DLC, build and test close Construction, performed in the same cycle by the same participants that generated the code. The objection is circularity, not thoroughness: tests derived by the same model, from the same specification, in the same context, encode the same reading of the requirement the implementation encodes, so passing them confirms internal consistency rather than external correctness. Benchmark evidence shows exactly this asymmetry: agent-generated code passes functional tests far more often than independently held security tests, and the gap does not close as models improve [8, 9]. Testing therefore becomes its own phase, with its own environment, hat and gate, working from the locked functional specification rather than the implementation; the unit of validation changes at this boundary, from the Bolt's task to Testing's assembled User Story, the smallest unit at which business value is observable.

The second is a distinct Maintenance phase with a diagnostic firewall. AI-DLC's Operations phase is deployment with monitoring, defining no mechanism for classifying post-deployment signals or routing

them to an owner. The signal is heterogeneous, a defect against an agreed specification, a request never specified, or evidence the specification was wrong, and treating all three as defects erodes the specification, since each undocumented accommodation widens the gap between what the system does and what the record says it should. A signal is therefore classified before any code is opened, and Context Reconstruction limits the developer to the specific report, the test evidence and the isolated fragment implicated, prohibiting whole-project context, a control for the reason developed in Section 6.6.2 [8].

The phases are listed in an order, but that order is not the shape of the process: Maintenance neither terminates the lifecycle nor patches in place. Its exit is always an entry into a named earlier phase, whose gate then applies in full: a business deviation routes to the Project Manager (PM) hat for a value decision and, if approved, re-opens Discovery and Requirements; a specification gap re-opens Design, Prototyping and Architecture with the technical specification amended and re-versioned; a production defect re-enters Implementation as a Fix-Bolt and returns through Testing. This is not Waterfall with a maintenance tail: in Waterfall, feedback is measured against a specification treated as frozen, whereas here the specification is exactly what is re-opened, re-approved and re-versioned, and code is regenerated from it rather than diverging from an unchanged one. What the framework refuses is the unnamed loop, a change applied to production code with no corresponding change to the specification it is supposed to satisfy, since that is how traceability is lost in practice. Figure 6.1 shows the six phases and these three loop-backs; Table 6.3 below is the detail behind each box.

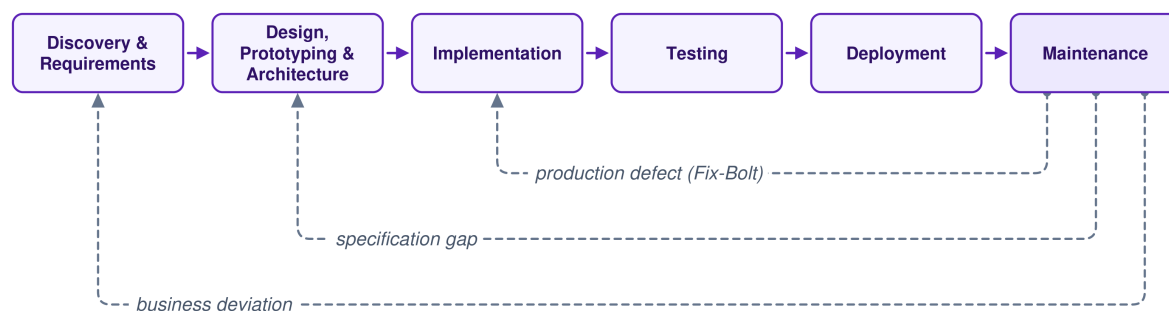


Figure 6.1: The six lifecycle phases in sequence, and the three named loop-backs out of Maintenance. Each loop-back re-enters a specific earlier phase, whose gate then applies in full - Maintenance never patches production code without a corresponding change to the specification.

Table 6.3: The six lifecycle phases and the preceding Strategic Alignment stage: permitted AI contribution, output, accountable hat, and the control that must be satisfied before each is exited. Each phase takes as input the output of the one above it.

Phase	Role of AI	Output	Human accountability	Mandatory control
Strategic Alignment (pre-lifecycle)	Assists market analysis; drafts candidate objectives and indicators	A Unit of Work carrying a stated business justification, Objective and Key Results (OKRs) and Key Performance Indicators (KPIs)	PM hat aligns the product vision with stakeholders and owns the justification	Stakeholder alignment and OKR sign-off before the Unit of Work reaches the Squad
Discovery & Requirements	Drafts fractional User Stories in natural language; compiles approved scenarios into formal Gherkin	Approved User Stories sized XS or S, with Gherkin acceptance criteria and stable scenario identifiers	The Trio validates correctness, completeness and business value through the Interactive Bridge	Mob Elaboration and the Discovery Gate
Design, Prototyping & Architecture	Proposes architectural options; generates User Interface (UI) structure and a formal technical specification	Approved prototype and an anchored technical specification (interface contract, data model, runtime contract)	Design Lead hat validates usability; Tech Lead hat validates feasibility, scalability and integration risk	Design Gate; no task enters the Bolt Backlog until both validations pass
Implementation	Generates and refactors code constrained by the Consistency Factor test set	Task-level code passing the pipeline and the Consistency Factor	Developer hat validates output, owns correctness and security, and must be able to explain the logic	Pre-Bolt architectural lock, the Bolt micro-cycle, and the Implementation Gate at story level
Testing	Drafts end-to-end Behaviour-Driven Development (BDD) scripts derived from the approved Gherkin	A story-level verdict and, on failure, the exact failing evidence	Quality Assurance (QA) hat verifies and refines generated scripts before execution and probes edge cases manually	The QA Validation Playbook and the Testing Gate, in a separate environment decoupled from the Bolt
Deployment	Answers the infrastructure delta question; generates configuration, Infrastructure as Code (IaC) or migrations only when required	A released increment and a persistent context updated to the live state	Whoever performs the deployment approves it and reviews its security implications	Deployment Gate, performed by the deploying hat; no separate gatekeeping team is assumed
Maintenance	Reconstructs a deliberately narrow context to explain one specific failure and proposes a bounded fix	A classified signal routed to the phase that owns it, with a verified diagnosis where applicable	Developer or PM hat classifies the signal and must verify any diagnosis before it is trusted	Diagnostic firewall (Context Reconstruction, whole-project context prohibited) and governed loop-back to a named phase

6.5 Roles and Responsibilities

Every role named is a hat: a function the framework requires to be performed, not a person who must be hired. One individual may wear several hats, and on a small team a hat may go unfilled, its responsibility falling to whoever performs the adjacent work. Table 6.4 summarises the hats and the gate each owns. Three reasons justify expressing responsibility as a function rather than a position: it fits a target population of anyone with a technical background working alone or in a small team, which a staffed-organisation role model would exclude; accountability expressed as a title lapses the moment the title is vacant, whereas a function survives reassignment; and the third is evidential, concerning what happens

when structural models are copied as blueprints.

The Trio, comprising the Product Owner (PO), Tech Lead and Design Lead hats, is inspired by the Squad concept popularised in the Spotify Model, and citing that model honestly requires citing its limits alongside it. It was published as a description of one company at one point in time, not a method for adoption elsewhere [43], its own principal author has since called it inspiration rather than a blueprint [44], and a former employee's first-hand account names two failure modes never resolved internally: unclear matrix decision authority, and autonomy without accountability [45]. The framework hardens against exactly these two. Matrix ambiguity is answered by the Tech Lead hat's architectural lock, fixing the technical boundary before any Bolt begins; autonomy without accountability is answered by separating logistics authority in the Bolt Master hat from technical authority in the Tech Lead hat, with every gate attached to a named function. A closer structural match is Team Topologies' stream-aligned team, which treats cognitive load as an explicit design constraint [46]: limiting developer agency over scope during a Bolt reduces the open decisions held while validating generated code, which is the activity Chapter 1 identifies as the new constraint. The PM hat, sitting outside the Squad, is structurally closer to the single-threaded leader Amazon adopted after its two-pizza structure stopped surviving scale [47], cited because even the most-copied structures are described as provisional by the organisations that ran them.

An earlier version named a DevOps Alliance, an external group approving releases and reviewing generated infrastructure code for security. It has been removed with no replacement, since naming a governance function without naming who holds it is the defect explicit responsibility exists to prevent: a body defined only by what it supervises can be assumed by the process to exist and never actually staffed. Both responsibilities remain mandatory; only the owner changed, to whoever performs the deployment, self-reviewed solo or peer-reviewed where a second developer exists. The framework prefers a control whose owner is unambiguous and occasionally the same person to one whose owner is impressive and frequently fictional.

The gates do not relax as the team shrinks. On a solo project each gate is self-checked, but what it requires is unchanged: the evidence in Table 6.5 must exist, be recorded, and be examined deliberately at the moment the gate is reached, not assumed to have been examined during the work that produced it. A gate's value lies primarily in forcing an explicit, separate act of validation against a written specification, and only secondarily in who performs it; where none is available it becomes a checkpoint rather than a handover, and the persistent specification is what makes self-review meaningful, since the reviewer checks the artefact against a record written before it existed.

Table 6.4: The hats: the function each discharges and the gate each owns

Hat	Function the framework requires	Gate owned
PM	Stakeholder alignment, OKRs and KPIs, and the business justification of a Unit of Work before it reaches the Squad; receives classified business deviations from Maintenance.	Strategic alignment sign-off
PO (Trio)	Business value and prioritisation of User Stories; leads the critique of AI-drafted scenarios during Mob Elaboration.	Discovery, with the Trio

Continued on next page

Hat	Function the framework requires	Gate owned
Tech Lead (Trio)	Technical specification, architectural feasibility and integration risk; decomposes stories into tasks and flags high-risk ones before any Bolt begins.	Design (technical) and the pre-Bolt architectural lock
Design Lead (Trio)	Prototype or visual element for every feature, and its validation through usability testing.	Design (usability)
Bolt Master	Continuous flow: prioritises the Bolt Backlog, manages dependencies, assigns by capacity once the Tech Lead hat has approved.	None; holds logistics authority deliberately separated from technical authority
Developer	Executes tasks with the model, generates and validates the Consistency Factor, and remains answerable for correctness, security and being able to explain the logic.	Implementation
QA	Validates the assembled story against the approved Gherkin in a separate environment, verifying generated BDD scripts before execution and probing edge cases manually.	Testing
Deploying hat	Not a distinct role: whoever performs a release approves it and reviews the security of any generated IaC or configuration.	Deployment

6.6 Governance Mechanisms and Quality Gates

6.6.1 Bolts as the Unit of Execution

A Bolt is the smallest execution iteration the framework recognises: the implementation of one task within a User Story, strictly inside the development environment and measured in hours rather than weeks. The term is adopted from AI-DLC [42]; what the framework adds is the sequence of obligations around it, since a short cycle without a fixed reference is a fast way to accumulate divergence.

Before any code is generated, the Tech Lead hat feeds the locked context (the approved Gherkin, the prototype and the technical specification) to the model, verifies the task list it proposes, and flags high-risk tasks. This **architectural lock** makes the technical boundary a matter of record rather than negotiation, letting tasks on either side of an interface proceed in parallel. The Bolt Master hat then assigns by capacity, giving developers the story's intent, not authority over the technical scope the lock has already settled. A Developer hat pulls one task, at which moment the Bolt begins and Bolt Cycle Time is measured from. Developer and model then produce the **Consistency Factor**, a strict automated test set derived from both specifications that constrains generation rather than reporting on it afterwards; the model generates against that constraint and the developer refines the result as an active editor, not an approver of a finished proposal. A task is done when the code passes the pipeline *and* the developer can explain the logic, since code no one can explain has been substituted for engineering rather than added to it. When every task is done, the story passes the Implementation Gate into staging.

The **Fix-Bolt** is the single exception: triggered by a failed Testing Gate or a verified defect, it omits task breakdown because the context already contains what that step would produce, the exact failing scenario and its evidence, and is justified by what is already known rather than a licence to skip planning for a change that merely feels small. Its outcome is routed by risk, where the risk-proportional principle becomes operational: a low-risk patch takes the **Fast Lane** directly to the delivery pipeline without re-entering formal QA, while a higher-risk one takes the **Secure Lane** back to staging for a Regression

Bypass, in which QA re-runs the existing suite before verifying the targeted patch.

6.6.2 Spec-Anchored Continuity

Spec-Anchored Continuity requires every AI-generated artefact to carry forward the context established by previously validated phases. The mechanism is a persistent versioned specification maintained alongside the codebase and split into two layers with different lifetimes. The **global layer**, the Memory Bank, holds what does not change between stories, namely architectural rules, technology-stack decisions, security policies and recorded workarounds for already-diagnosed defects; it is written rarely, amended deliberately, and applies to every generation on the project. The **local layer**, the Feature Specs, holds what is specific to the work in hand, namely the approved Gherkin acceptance criteria and the technical specification produced at the Design Gate; it is written once per story, locked at a gate, and amended only through a recorded versioned amendment. The split is what makes the mechanism usable, since a single undifferentiated context grows monotonically until it contains more irrelevant material than relevant, reintroducing the problem it was meant to solve.

The central argument is that narrowing the model's working context is a control rather than a convenience: a convenience makes a correct outcome easier to reach, whereas a control is a measure whose absence makes an incorrect outcome undetectable. Anchoring satisfies the second definition on three counts: it supplies a reference that existed before the generated artefact did, the precondition for validation being possible at all; it bounds what the model may act upon, an unbounded context being the condition under which a model produces a wide, confident and substantively wrong change, which is why whole-project context is prohibited during Maintenance diagnosis [8]; and because both layers are versioned and every post-lock edit is a dated amendment, divergence between what was agreed and what was built becomes a detectable event rather than an unattributable drift. This is the spec-driven-development instantiation already named in Table 6.2; a recent preprint independently argues, from a multivocal review, that specification discipline rather than model capability is the binding constraint on dependability [41]. The contribution is not the idea of anchoring but the specification of where it is mandatory, which gate locks each layer, and what constitutes a legitimate amendment.

6.6.3 Quality Gates

A quality gate has three named components: an owning hat, a specific body of evidence that must exist and be examined, and a defined consequence when it is absent or inadequate. A checkpoint missing any of the three is a ceremony rather than a control, since it can be passed without anything being established. Table 6.5 specifies the five gates on those terms. One property of the set is worth stating separately from any individual gate: failure never discards work and never silently degrades it, but returns the work to a named earlier activity with the evidence attached, which is the same routing discipline Maintenance applies to post-deployment signals.

Table 6.5: The five quality gates: owner, required evidence, and the consequence of failure

Gate	Owning hat	Evidence required to pass	Consequence of failure
Discovery	The Trio	Testable Gherkin acceptance criteria approved through the Interactive Bridge; every story sized XS or S; the functional specification locked and versioned.	Scenarios return to Mob Elaboration; oversized stories are decomposed before re-submission.
Design	Design Lead and Tech Lead hats	An approved prototype validated by usability testing; a technical specification covering interface contract, data model and runtime contract, anchored into the persistent context.	Feedback returns to prototype generation or architectural specification; no task may enter the Bolt Backlog.
Implementation	Developer hat	Every task complete, passing the pipeline and the Consistency Factor, with the developer able to explain the generated logic.	The story does not enter staging; incomplete or unexplained tasks return to the Bolt Backlog.
Testing	QA hat	The assembled story validated in staging against the locked Gherkin through verified BDD scripts, with edge cases probed manually.	A Fix-Bolt is triggered with the exact failing evidence attached; the story re-enters staging afterwards.
Deployment	Whoever performs the deployment	Explicit approval of the release and a security review of any generated IaC, configuration or migration; the persistent context updated to the live state.	The specific objection is fed back for a compliant regeneration; the release does not proceed.

6.6.4 Governance Metrics

Three governance metrics are defined, chosen to measure whether the collaboration is safe and whether it is flowing rather than to measure output volume, since output volume is the quantity Chapter 1 shows rising without a corresponding improvement in delivery [12]. **Bolt Cycle Time** is the elapsed time from a task being pulled as a Bolt to its being marked done, defined at task level rather than story level because a story-level measurement conflates execution with the queueing, review and gate transitions surrounding it, and the framework's claim about continuous flow is a claim about execution. **Context Traceability Rate** is the proportion of deployed Bolts and stories whose context chain is complete, meaning the deployed artefact resolves back to a locked technical specification, to the functional specification it derived from, and to the Unit of Work that motivated it; it measures whether Spec-Anchored Continuity is holding in practice as opposed to being prescribed. **AI Defect Escape Rate** is the proportion of production defects originating in AI-generated logic that passed the Testing Gate, and it is the metric that detects a gate which has become ceremonial, since a gate satisfied without being exercised shows up here and nowhere else.

These are expressed in vocabulary continuous with established AI governance standards rather than bespoke to this work: the National Institute of Standards and Technology (NIST) AI Risk Management Framework's Govern, Map, Measure and Manage [48] correspond respectively to this chapter's design principles, phase mapping, metrics and gates, and ISO/IEC 42001:2023 offers a complementary plan-do-check-act vocabulary for the same loop [49]. This is a claim about shared language only: neither the framework nor Apex has been audited against either standard, and nothing here asserts compliance

with or intent to certify against them, only that these constructs occupy the conceptual space standards bodies are converging on.

6.7 Work Organisation

The framework replaces the familiar hierarchy of epics, sprints and sprint backlogs with continuous flow, the most disruptive single element of the proposal and therefore the one requiring the strongest, empirical justification. Sprint cadence manages a world in which implementation is the binding constraint: a fixed-length iteration lets a quantity of implementation work be forecast, committed to and inspected at a boundary, calibrated to how long a useful amount of implementation takes. The evidence in Chapter 1 indicates that premise no longer holds where AI assistance is heavy: developers in a randomised controlled trial were measurably slower with assistance while believing they had been faster [11], and telemetry across 1,255 teams pairs the merged-pull-request increase reported there with pull requests 154% larger, 91% longer review times, and no significant correlation with delivery outcomes [12]. The constraint moved rather than disappeared: change volume rose, review lengthened, delivery stayed flat, so a cadence calibrated to the old constraint now obscures the new one. The structural response has two parts: execution compressed into Bolts, so the queue in front of validation refills in small increments rather than batches, and validation decoupled into its own phase, environment, hat and gate, so the constraint is scheduled and measured in its own right. Both are falsifiable: if review is not the constraint in a given setting, Bolt Cycle Time and the QA queue will say so.

Three levels of work are retained, and the vocabulary of the upper two follows AI-DLC [42]. A **Unit of Work** is the core business problem, owned by the PM and PO hats, the level at which business justification is argued and acceptance criteria are generated as a group; grouping them this way rather than requirement by requirement is itself supported by evaluation, an epic-organised pipeline outperforming a requirement-aligned baseline across 107 requirements on expert-rated correctness (4.61 against 4.14 out of 5) and completeness (4.31 against 3.50) [50]. **User Stories** are the tangible deliverables, constrained to extra-small or small, the level at which QA tests because they are the smallest unit at which business value is observable; the size constraint is load-bearing, since a story describable by a handful of scenarios is small enough for a reviewer to hold entirely in mind while validating generated code against it. **Tasks** are the granular technical steps executed as Bolts, entering the **Bolt Backlog**, a continuous prioritised queue, only after decomposition with the model, Tech Lead verification, and risk flagging. Developers have limited agency over task creation and selection deliberately, not out of distrust: the decisions withheld are exactly those the architectural lock has already settled, for the cognitive-load reason argued in Section 6.5. The team's project management tool is expected to expose states corresponding to the phases as guidance rather than a literal requirement; what matters is that each column maps unambiguously onto one phase.

6.8 Operational Playbooks

The phases, gates and hats state what must hold; the six playbooks in Table 6.6 state how the recurring procedures are performed, each with its trigger, its participants and its termination condition. Together

they are the operational content of the framework: a team that follows these six and observes the gates in Table 6.5 is operating the framework, irrespective of the tooling it uses.

Two of the steps are substantive design choices rather than sequencing. In Maintenance, the prohibition on whole-project context is the single most restrictive rule in the framework, for the reason given in Section 6.4. In the Fix-Bolt, the Regression Bypass re-runs the existing BDD suite rather than regenerating it, because a regenerated suite is a different measurement instrument, and comparing a patched system against it cannot distinguish a fixed defect from a changed test.

Table 6.6: The six operational playbooks: trigger, procedure, and termination

Playbook	Trigger	Procedure	Terminates when
Mob Elaboration	A Unit of Work brought to the Squad	The Trio supplies intent, personas and domain constraints; the agent drafts fractional stories in natural language; the Trio cross-examines for value, feasibility and coherence, issuing confirm, add, delete or modify instructions until satisfied; the agent then compiles the approved scenarios into Gherkin and anchors the result.	The Discovery Gate passes and the functional specification is locked
Design and Prototyping	Locked Gherkin acceptance criteria	The Design Lead hat prompts for screens and flows; the Tech Lead hat prompts for interface contract, data model and runtime contract and anchors them; usability testing and architectural review run in parallel.	Both validations pass and tasks enter the Bolt Backlog
Maintenance and Evolution	A post-deployment signal	Classify before any code is opened: a business deviation or specification gap routes to the PM hat for a value decision and re-opens the owning phase; a technical defect routes to a Developer hat, who narrows the context to the report, the test evidence and the isolated fragment, asks only why that logic failed, and verifies the diagnosis manually.	The signal is classified and routed, with a verified diagnosis where applicable
Fix-Bolt	A failed Testing Gate or a verified defect	Pulled immediately with task breakdown skipped; the agent resolves only the specific defect while adhering to the anchored specification; the developer reviews for newly introduced vulnerabilities, not only symptom resolution; existing and new unit tests must pass.	Low risk takes the Fast Lane to the pipeline; higher risk takes the Secure Lane through a Regression Bypass in staging
QA Validation	A story that has passed the Implementation Gate	The QA hat opens the locked functional specification rather than the implementation, prompts for end-to-end BDD scripts and verifies them manually <i>before</i> execution, since an unverified script can report a passing scenario it never exercised, then probes edge cases exploratively.	The Testing Gate passes, or a Fix-Bolt is triggered with the failing evidence
Deployment and Release	A story that has passed the Testing Gate	The agent answers one bounded question, whether the release requires new infrastructure, configuration or migrations, and generates artefacts only when it does; whoever deploys reviews any generated artefact for vulnerabilities and policy violations.	The Deployment Gate passes and the persistent context is updated to the live state

6.9 Positioning Against Alternatives

A design is only defensible against named alternatives. Four are engaged below, each stated at its strongest, credited with what it genuinely does better, and met with the evidence motivating a different choice. The framework's constructs are an original synthesis; the sources cited throughout ground individual pieces rather than forming a joint blueprint this work reproduces.

No framework at all. The premise is that model capability suffices and process overhead is pure cost. Conceded: no coordination cost, no artefacts to maintain, no gate that can turn ceremonial, and for a short-lived prototype with no maintenance horizon it is very likely correct. Its failure mode is documented rather than hypothetical: the large majority of functionally correct agent output was found insecure, carrying substantially more vulnerabilities than human-written equivalents [8, 9]. The decisive detail is not either figure's size but that security performance did not improve as functional correctness did; if it were lagging capability, waiting would be a valid strategy, but it is flat, which turns validation from discipline into a structural requirement.

Adopt AI-DLC as published and unmodified. The premise is that a coherent three-phase model already exists from a credible source and extending it adds complexity without control [42]. What it does better is real: simpler, operated at commercial scale, less process surface for a small team. The objection is narrow, the two gaps argued in Section 6.4: build and test folded into the close of Construction, and an Operations phase with no published mechanism for classifying post-deployment feedback. The divergence fills gaps its own documentation leaves open rather than reinventing what it already solves, and its reported productivity outcomes remain vendor-reported and unverified, so adopting it unmodified cannot be justified by evidence of results.

Keep Scrum or SAFe cadence and treat AI as a faster development environment. The premise is that existing ceremonies already provide planning, review and retrospection, and AI only changes the speed of one activity. What it does better is not trivial: the cadence is understood, tooled, compatible with planning cycles, and costs nothing in retraining. The evidence against it is the strongest available here because it is a controlled experiment, not survey data: developers estimated afterwards that assistance had made them substantially faster when measurement showed it had made them slower [11]. The retrospective this alternative relies on to catch that is a weak instrument for catching it, while an iteration boundary calibrated to implementation keeps reporting on implementation while the constraint sits in review [12].

Metrics-only governance, with no phase structure. The premise is that an organisation can track the right capabilities and outcomes, publish them, and let teams self-organise. This is the most serious of the four: far cheaper to adopt, presumes no lifecycle shape, adapts to context, and its own literature legitimately warns that no single metric can stand for the whole and any measure elevated to a target stops being a good one [51], a warning this framework takes as seriously as any other. The response is a distinction, not a rebuttal: metrics measure whether governance is working, they are not themselves a governance mechanism. A dashboard can accurately report rising review time and escaping defects while containing no mechanism for what a model may act upon, no point where an artefact must be checked against a prior specification, and no rule for where a post-deployment signal is routed. The two are complementary, which is why the framework defines its own metrics in the same vocabulary rather than proposing an alternative to measurement.

6.10 Summary

DRQ1 asked how AI-supported activities can be mapped to SDLC phases so the point, purpose and boundary of AI participation are unambiguous. The answer is the six-phase model of Section 6.4 with the Strategic Alignment stage preceding it, and the mapping in Table 6.3, fixing for each phase what enters, what the model may contribute, what leaves, which hat is accountable and which control must be satisfied before exit. The boundary is unambiguous in two ways a phase list alone would not achieve: the model is constrained at every phase to that phase's anchored specification rather than the whole project, and every phase terminates in a human act of acceptance recorded against named evidence. The map extends its closest precedent at two evidenced points: a decoupled Testing phase, and a Maintenance phase with a diagnostic firewall.

DRQ2 asked which governance mechanisms, gates and human-in-the-loop practices keep AI contributions traceable, validated and accountable. The answer is Section 6.6's set: Bolts as the governed unit of execution, with the Consistency Factor constraining generation rather than assessing it afterwards; Spec-Anchored Continuity as a two-layer versioned specification making context-narrowing a control rather than a convenience; five gates each with a named owner, evidence and failure consequence; and three metrics in vocabulary continuous with established governance standards, without any claim of compliance. Accountability attaches to functions rather than positions so it survives a team of one, and every failed gate routes work back to a named phase rather than discarding it.

Two limitations belong here, not as a later concession: spec-driven development, on which Spec-Anchored Continuity rests, is characterised by its own principal source as emerging with unsettled scaling behaviour [40], and the framework's constructs are an original synthesis argued from evidence but not yet demonstrated in use. Chapter 7 presents Apex, the reference implementation, and reports which constructs were implementable as specified, which were weakened or proxied, and which resisted implementation, the substance of DRQ3.

7

Proposal Implementation

Contents

7.1 Purpose and Positioning	39
7.2 Architecture	40
7.3 Operationalising the Framework	44
7.4 Design History and Practitioner Feedback	48
7.5 Quality Assurance of the Implementation	50
7.6 Limitations of the Implementation	51

The framework presented in Chapter 6 describes a methodology. This chapter presents Apex, the working software system that instantiates it, which serves both as evidence that the framework’s governance constructs are implementable and as the vehicle through which the framework is demonstrated and evaluated in Chapters 8 and 9. In *DSRM* terms the two are distinct artefacts produced by the same Design and Development activity: a method and its instantiation. This chapter answers *DRQ3*, and answering it honestly means reporting which constructs resisted implementation as well as which did not.

7.1 Purpose and Positioning

Apex is not a coding agent and is not positioned against one. A coding agent accelerates the writing of code; Apex governs whether the code that results still answers the question that prompted it. The two are complementary, and the distinction is the whole reason the system exists: acceleration without anchoring

is precisely the condition Chapter 1 shows moves the bottleneck to review rather than removing it. Every artefact Apex produces carries a stable specification identifier back to the requirement it derives from, so divergence between what was agreed and what was built becomes a detectable event rather than a gradual drift that nobody is positioned to notice.

What Apex deliberately does not do is as much part of its definition. It does not train or fine-tune models, and treats the model as a third-party capability that may change beneath it. It does not provision infrastructure: where a release needs configuration or migration scripts it generates them for a human or a Continuous Integration (CI) pipeline to execute, and never applies them itself. And it does not replace the project management tool a team already uses. It sits above that tool, reads and writes through it, and expects the team's existing board to remain the system of record for work items, which is why the framework's board guidance in Section 6.7 asks only that a team's own columns map onto framework phases rather than that they be renamed.

7.2 Architecture

Apex is a split full-stack application. The frontend is Next.js 15 with the App Router in TypeScript; the backend is Python 3.12 with FastAPI; the two communicate over a typed REST API. Server state on the frontend is owned by React Query and client state by Zustand, with credentials persisted only to session storage so that they clear when the tab closes. Backend routers are kept deliberately thin, with workflow logic held in a service layer and a schema for every request and response boundary. That separation is an engineering choice rather than an accident of growth: it keeps route handlers testable as thin adapters and workflow logic testable in isolation from Hypertext Transfer Protocol (HTTP) concerns, which is what made the test strategy in Section 7.5 possible at the sizes it reaches. Figures 7.1 and 7.2 show the arrangement.

Four properties of that architecture carry framework constructs rather than merely supporting them, and each is described in turn.

Spec-anchored storage. All workflow state lives in Markdown context files together with a machine-readable story index, held under a path keyed first by instance and then by project. The instance key isolates tenants, so a hosted project management account and a privately deployed instance of the same tool are fully separate tenants even for the same human user; the project key isolates individual projects within a tenant. Storage is an abstraction over either local disk or cloud file storage, and the same code path runs in both, which the test suite verifies by pinning the abstraction to local disk rather than replacing it with a stand-in. This is Spec-Anchored Continuity made concrete: the specification is a set of files that outlive any session and that every generation step is constrained by. Access to those files is required to pass through a single service, never a direct import from a route, because that service is the one place where per-request project isolation is enforced. That convention is treated as an architectural invariant rather than a style preference, for reasons Section 7.5 makes concrete.

A deliberate single-writer concurrency model. The backend was originally pinned to one replica and one worker. The story index and workspace configuration live on shared file storage guarded only by a process-local lock, so a second replica would not crash but would silently race on writes, which is worse, because a lost update looks like success. Threads still interleave within the one process, so an in-process lock remains necessary and is used consistently. This was a documented constraint rather

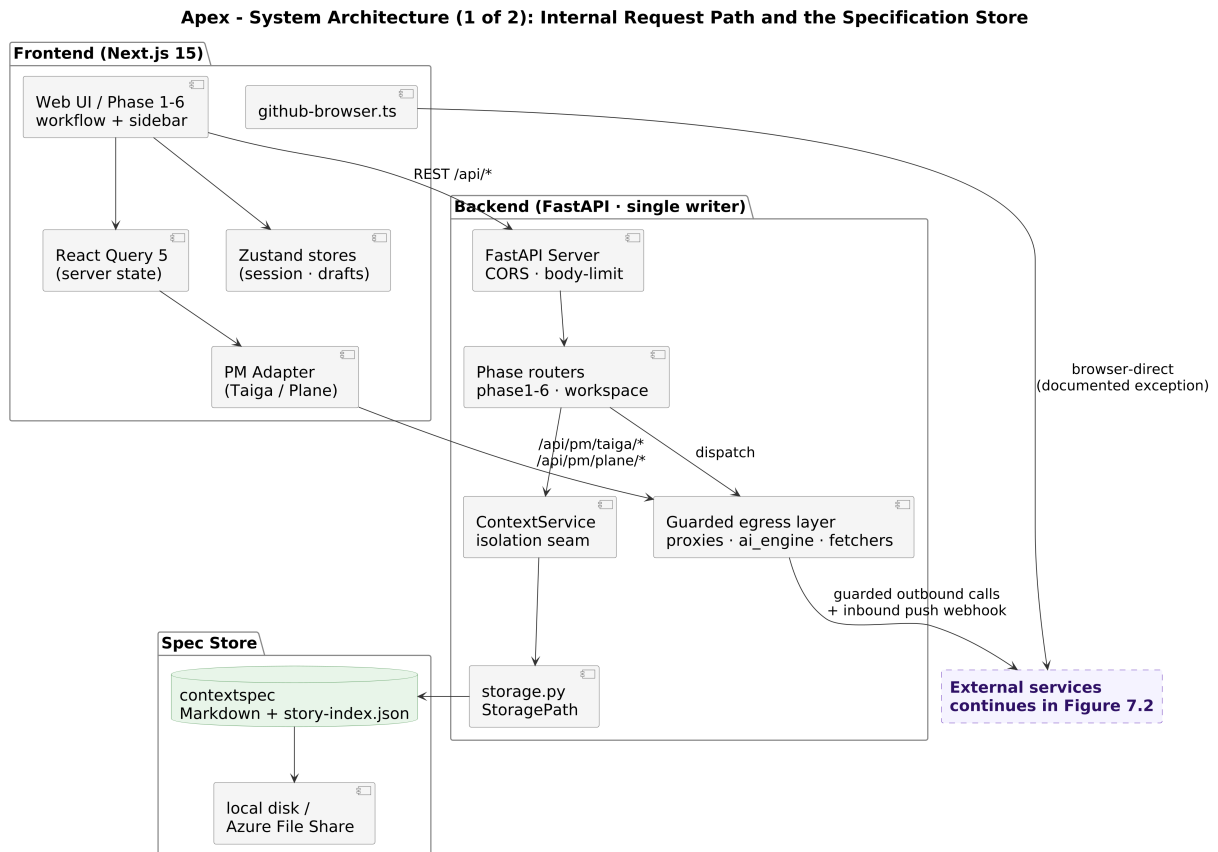


Figure 7.1: Apex's system architecture, part one of two: the frontend, the backend and the specification store. The figure is split across two pages for legibility; the point of it is the position of two boundaries rather than the inventory of components, and this half shows the first, where all workflow state is written to a per-tenant, per-project specification store rather than to an application database. The backend's guarded egress layer is drawn collapsed to a single component here and expanded in Figure 7.2; the dashed connector on the right marks where the two halves join.

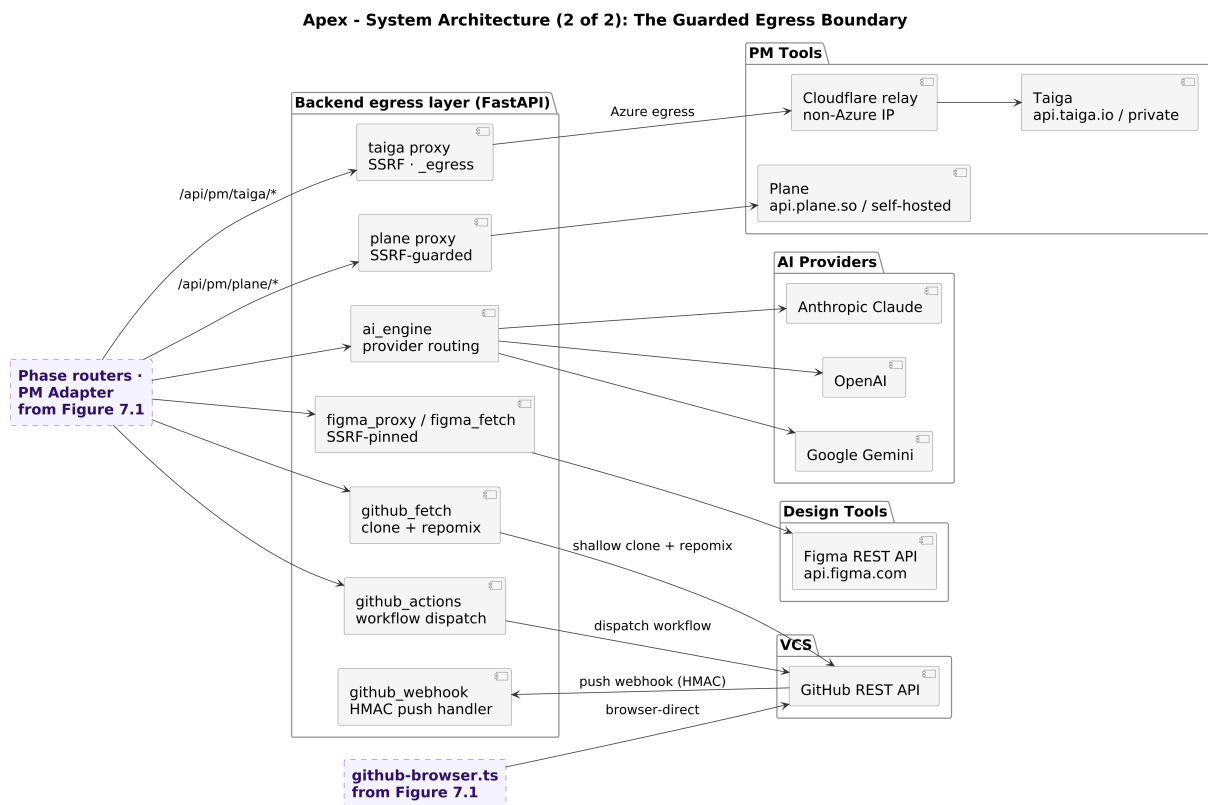


Figure 7.2: Apex's system architecture, part two of two, expanding the guarded egress layer of Figure 7.1 and the external services it reaches. This half shows the second boundary the figure exists to make, that every outbound call to a user-configurable host passes through a server-side guarded proxy. The dashed connectors carry over the callers from Figure 7.1; the browser-direct path to the version control host is the single documented exception, and the credential for it never reaches the backend.

than an oversight, and it came with a stated precondition for lifting it. That precondition was subsequently built and enabled: an optional coordination layer promotes the locks to cross-replica equivalents when a coordination service is configured, and falls back to byte-identical single-replica behaviour when it is not. It has been running in production with three replicas since June 2026, verified by inspecting live client connections rather than by reading a startup log line.

Proxy-first external integration. Every outbound call to a host the user can configure is proxied server-side rather than issued from the browser, specifically to close server-side request forgery: the proxy validates that the target uses Hypertext Transfer Protocol Secure (HTTPS) and resolves to a non-private address before forwarding, and caps the size of any body it relays. The identity anchor a request is validated against is derived from the same validated host, which is what makes the storage isolation above trustworthy rather than merely tidy. One integration is a deliberate exception, since the provider permits direct browser access and the credential never reaches the backend for those calls; two operations within it are not exceptions, because they need a server-side filesystem and a server-held credential respectively.

The project management adapter. Apex does not own the board; it reads and writes one the team already has, so the tool it sits above is an integration rather than a component. Every operation passes through a single adapter interface, and the system was built against Taiga first, with Jira and Plane added subsequently behind the same interface. Plane matters beyond parity because it is the tool the partner organisation in Chapter 8 is moving to, so support for it was driven by a real external requirement rather than by a wish to claim breadth.

Adding a second and third tool is the closest thing this work has to a test of the framework's claim, in Section 6.2, that it names the property a mechanism must have rather than the product providing it. The claim survived, but not without cost, and what cost it is informative. The two tools differ in authentication shape, one exchanging credentials for a token and the other using a pre-issued key, which was absorbed by generalising the identity anchor rather than by branching at every call site. They differ in identifier type, integers against Universally Unique Identifiers (UUIDs), which the adapter reconciles by minting a synthetic identifier and carrying the real one alongside, rather than widening a type used in more than thirty places. And three of the second tool's Application Programming Interface (API) limitations are permanent rather than missing work: there is no page update or delete endpoint, no optimistic-concurrency field, and no server-side label filter. Each was mitigated at application level, by versioning republished pages, by carrying a timestamp as an advisory version that feeds the existing conflict-retry path, and by caching a label list, and none of the three is a fix. The general finding is that an abstraction over project management tools holds at the level of operations and fails at the level of guarantees: what a tool lets you do can be made uniform, while what it promises about concurrency and identity cannot.

Provider abstraction. The model provider is detected from the model identifier and routed accordingly, so no other part of the codebase branches on which vendor is in use, and provider errors are mapped to typed exceptions at a single boundary so that every route inherits consistent semantics without repeating the mapping. The tool-independence claim tested above against project management tools therefore holds for model vendors as well, and at lower cost, since a provider offers no guarantees about concurrency or identity for an abstraction to leak.

7.3 Operationalising the Framework

This section answers DRQ3, which asks how far the framework's governance constructs can be operationalised in a working system and which of them resist it. Table 7.1 states the mapping construct by construct, and every row points at something that exists in the codebase rather than something planned. Three outcomes are distinguished and the distinction is the substance of the answer: a construct realised as specified, a construct realised as a proxy that measures something adjacent to what the framework defines, and a construct that resisted implementation and is not present. Figures 7.3 and 7.4 show how a practitioner actually walks the phases those constructs govern.

Table 7.1: Framework constructs against the mechanisms implementing them in Apex, and the outcome of each

Construct	Implemented mechanism	Outcome
The six lifecycle phases	One workflow surface per phase, each gated on the preceding phase's exit condition rather than freely navigable.	Realised
Quality gates	A per-story phase-status flow advancing through gherkin_locked, design_locked, implementation, qa, qa_passed and deployed; middleware refuses phase access when the status does not permit it.	Realised
Spec-Anchored Continuity	Versioned Markdown context files plus a story index under a per-tenant, per-project path, split into project-wide files and per-story specifications.	Realised
Stable specification identifiers	Epic, entity, screen and scenario identifiers minted at generation time and recorded in a specification index, so every later artefact resolves back to a requirement.	Realised
The Interactive Bridge	Phase 1 drafts scenarios in natural language for confirm, add, delete or modify instructions, and compiles to Gherkin only after approval.	Realised
The Consistency Factor	A generated test set produced before implementation and carried into the developer pack for each task.	Realised
Bolts and the Fix-Bolt	A per-task micro-cycle recording pack_ready, pushed and done transitions with timestamps, independent of the story's own phase status.	Realised
Bolt Cycle Time	Elapsed time from pack_ready to done per task, aggregated as median and 90th percentile. Initially computed from story-level gate transitions and presented under the same name, which measured queueing and review rather than execution; corrected to the task-level definition the framework specifies.	Realised, after a proxy was found and replaced
Context Traceability Rate	An aggregate over deployed stories, computed since June 2026; the earlier claim in this row that no aggregate was computed was itself in error, not a later change to the system. Strengthened in September 2026 to require the chain have no live break - no unresolved backward-trace flag and no unacknowledged conformance regression - rather than merely that every artefact file exists.	Realised, initially misreported here as unimplemented

Continued on next page

Construct	Implemented mechanism	Outcome
AI Defect Escape Rate	A proxy added in September 2026: the proportion of deployed stories with a project-management issue confirmed as a bug, linked to the story, and dated after its deployment. A defect reported outside the project-management tool, or reported without being linked, stays invisible, and the system still cannot attribute a defect to generated logic specifically.	Realised as a proxy
Spec-drift detection	Built for the whole specification and removed (Section 7.4); the technical specification was one project-wide file, so an edit could not be attributed and any change flagged every story at or past design lock - one real edit flagged 44 stories at once. Revived in September 2026 for the functional specification alone, where each scenario already carries a stable identifier naming its story, so an edit is attributed to exactly the stories whose scenario changed. The technical specification has no equivalent per-item ownership and was not revived.	Partially realised, resisted at the technical-specification granularity
Server-side reconciliation of generated output	Coverage claims and task dependency identifiers are re-checked against the parsed specification rather than trusted as reported, with unmatched claims dropped.	Realised, and added because it was needed

Four of those rows deserve emphasis because they are the ones that answer DRQ3 rather than merely illustrating it.

The **Bolt Cycle Time** row is the clearest instance of a proxy being mistaken for the construct. The framework defines the metric at task level precisely because a story-level measurement conflates execution with the queueing, review and gate transitions surrounding it. The first implementation measured story-level gate transitions and displayed the result under the framework's own name for the task-level quantity, which is exactly the substitution the framework warns against elsewhere. It was corrected, but the fact that it happened at all is evidence for a general point: a construct can be implemented, displayed and believed while measuring something else, and only comparing the implementation against the written definition catches it.

The **Context Traceability Rate** row corrects this chapter's own earlier account rather than reporting a later change to the system: the metric had already been computed as an aggregate over deployed stories since June 2026, so the original claim that no aggregate was computed was simply wrong. The error is informative in the same way Section 7.5 treats an unverified self-report: a claim about the system's state is not evidence of that state until it is checked against the code, and this chapter's first draft of the claim was not checked closely enough. The strengthening the row records closes a separate gap, where a story could show a complete chain while Phase 6 had already flagged it broken.

AI Defect Escape Rate moved from resisted to realised as a proxy in September 2026. The system still cannot observe a production defect directly or attribute one to generated logic, the limitation Section 7.6 states and which remains true. What changed is that the project-management tool already in use turns out to carry a usable signal Apex was not reading, the one the row describes; it measures reported, linked, post-deployment defects rather than defects in general. That the construct resisted the system's strongest form of observation and was still operationalised at a weaker one worth having is the distinction Section 7.3's three-outcome taxonomy exists to keep visible, applied here to itself.

Spec-drift detection is the construct that resisted implementation most instructively, and remains only partly resisted. The framework assumes the specification is scoped per story, which is what makes

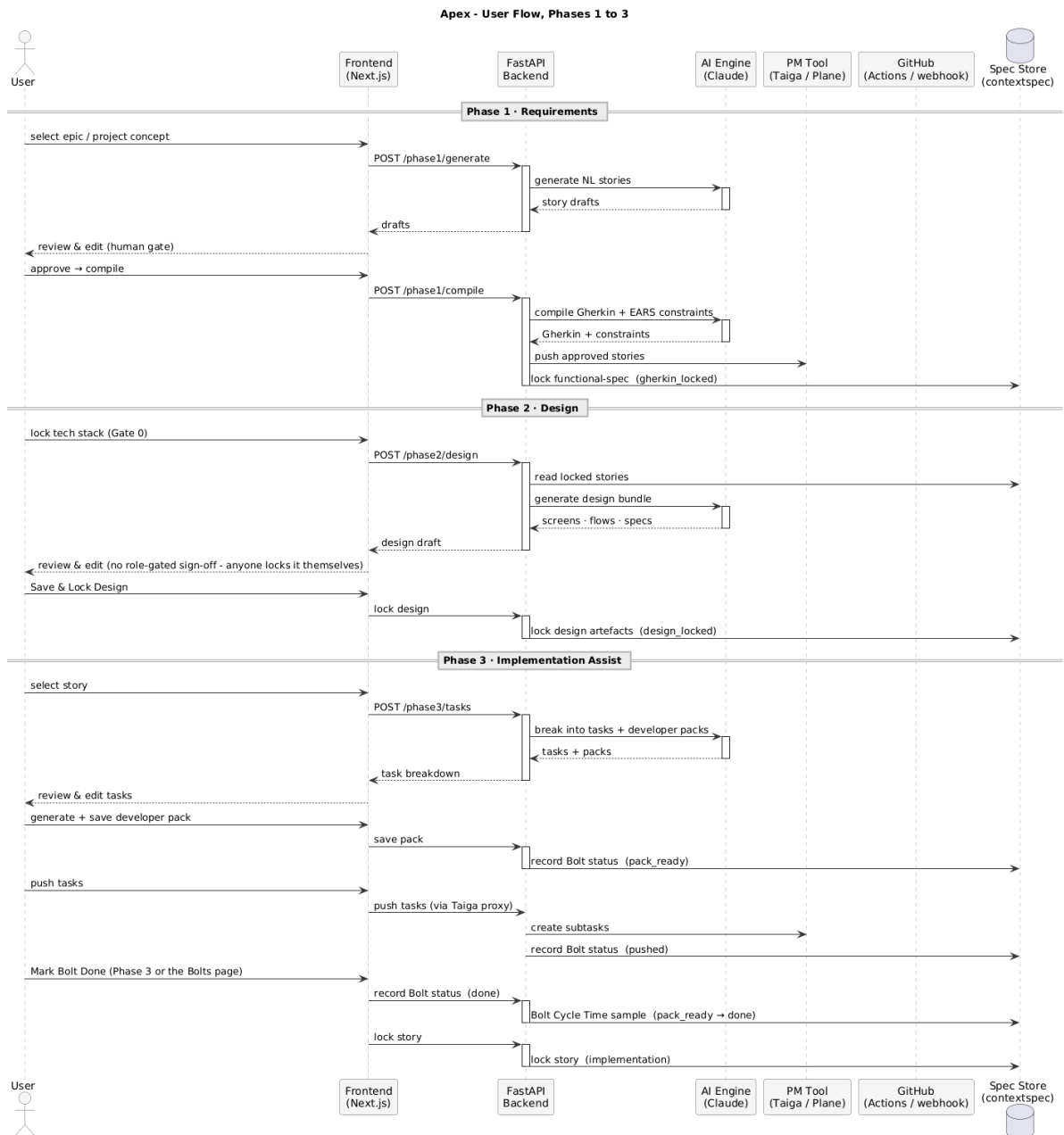


Figure 7.3: A representative walk through Apex, Phases 1 to 3. The figure is split across two pages for legibility; what it is meant to show is that every phase terminates in a write to the specification store and a human acceptance, so no phase can be exited by generation alone.



Figure 7.4: The same walk, Phases 4 to 6, continuing Figure 7.3. Note the loop-backs out of Testing and out of Maintenance: neither returns to an undefined state, and each re-enters a named phase whose gate then applies in full.

an amendment attributable; the technical specification did not meet that assumption, and the cascade the row records is the consequence. Removing it was the right call, but the finding is not that the feature was hard: it is that a framework construct silently depends on a granularity assumption the framework never states explicitly, and the instantiation is what exposed the omission. That is formative evaluation feeding back into the method, and it is recorded in Section 7.4 as such. The functional specification, unlike the technical specification, already meets that assumption, since each scenario is written under its own story's heading and carries a stable identifier naming that story, which is what made the September 2026 revival possible there. The technical specification was not revived the same way, since no equivalent per-item ownership exists anywhere in the system, and inventing one here would trade the original mistake for its mirror image - false precision in place of false breadth.

7.4 Design History and Practitioner Feedback

Feedback gathered while building Apex changed the framework, not only the tool. Recording that path is what makes the DSRM iteration explicit rather than implied, and it is reported here as observation, change and rationale rather than as a list of releases. Changes that were made and later reverted are included, because a reverted change is evidence about the design rather than an embarrassment to be omitted.

Two changes were framework-level and are the clearest evidence that evaluation fed back into the method. The first was the **removal of a role**. An earlier version of the framework named a DevOps Alliance, an external group holding deployment approval and the security review of generated infrastructure code. Feedback observed that the framework was intended for anyone with a development background, working alone or in a small team, and that a role defined only by what it supervises can be assumed by the process and never staffed by the organisation. The role was removed and its two responsibilities reassigned to whoever performs the deployment, self-checked where the team is one person. The rationale is argued in Section 6.5; what matters here is that the change originated in observation of use rather than in analysis.

The second was the **demotion of a mandatory step**. Phase 2 originally required a separate sign-off from named design and technical leads before design could be locked. The same observation applied: role-gating a gate presumes a staffed organisation, and on a solo project it either blocks the work or is satisfied by the same person twice, which teaches the practitioner that gates are ceremonial. The step was removed and the gate redefined against recorded evidence rather than against a second signature, which is the form it takes in Table 6.5. The framework's position that a gate's value lies primarily in forcing an explicit act of validation, and only secondarily in who performs it, is a direct consequence of this iteration.

Three tool-level changes are worth recording for the same reason. A **design-conflict detector** was built, then found to flag conflicts that were not conflicts, since two stories in the same epic legitimately share files; it was narrowed to cross-epic file overlap and endpoint collision, and later removed entirely. **Spec-drift flagging** was built, cascaded as described in Section 7.3, and was removed while its underlying audit trail was kept. An **authentication flow** for one design integration was implemented and then deliberately reverted, because it required an operator to register an application and hold a client secret in backend configuration, which is incompatible with the zero-configuration setup the tool otherwise of-

fers; the simpler token-based path was restored as the final decision rather than left as an unfinished half-feature.

The nature of that feedback should be stated plainly, because it bears on how much weight the design history can carry. It came from five practitioners consulted individually during construction, one of whom was consulted a second time after the framework had been substantially revised in response to the first round. The sessions were informal: no interview guide, no fixed question set, no recording, and no demographic data collected. Feedback was captured in the author's own contemporaneous notes as discrete items, each associated with the person who raised it, which is how the changes below can be traced. Participants are not identified here and no table of dates or participant profiles is given, because the sessions were not conducted under any expectation that they would be reported that way. The record is therefore a participant's notes rather than an independent transcript, and it carries the selection that implies: an item the author did not think worth writing down at the time is not in it.

What follows from that is a real limit and it belongs here rather than in Chapter 9. Because each item is attributable, *what* was raised and *what* changed in response are both verifiable against the codebase and the framework document. What cannot be recovered is how strongly any item was held, whether an observation was one person's reaction or a view several would have shared had they been asked, and what the practitioners who did not raise a given point thought of it. The design history is therefore evidence that the method was revised in response to use, which is the DSRM claim it is offered for, and it is not evidence about the distribution or strength of practitioner opinion. Three of the five practitioners consulted here later also contributed, separately and with their own consent for that use, to the semi-structured interviews reported in Section 9.3.4; the two sections draw on different material from some of the same individuals rather than reporting the same evidence twice, and an item appears in only one place. What the structured interviews in Chapter 9 add is exactly what this section cannot supply: a written guide, a stated (if under-target) participant profile, and a thematic analysis method applied consistently rather than notes kept only where a design change followed.

The second round is worth isolating, because it produced the sharpest challenge the framework received and the one that changed the shape of this dissertation rather than the shape of the tool. Its substance was that the work was being presented as a product and would therefore be judged as one, against the question of what it offers that a general-purpose coding agent does not; that the scientific questions, why this framework rather than another and how it maps to a given organisation, were being left implicit; and that the correspondence to the published models it draws on should be stated step by step rather than asserted. The response is distributed across three chapters rather than confined to a change log. Section 7.1 answers the coding-agent question directly, Section 6.9 engages four named alternatives rather than defending the design in the abstract, Section 6.4 argues the two points of divergence from its closest published precedent instead of claiming a one-to-one mapping, and Chapter 8 adopts the reframing that the instantiation is itself a demonstration of the method. That a single round of feedback is traceable into four sections of the final document is the clearest evidence available that this was formative evaluation rather than consultation after the fact. This second round is reported only here, in full, rather than split with Section 9.3.4; its impact was on the framework's own construction and argument, which is this section's subject, not on the distribution of practitioner opinion, which is Section 9.3.4's.

A sixth, later consultation is worth naming separately, and is not part of the five above. An Agile

Coach was given a live presentation of the framework and a demonstration of Apex; the reaction, and the two open gaps it named in the tool's interface and in the coach's own organisation's AI governance, are reported in full in Section 9.3.4 as this study's Agile-expert interview rather than duplicated here. It shares the limits stated for the five practitioners above, and is separated from them because it took place later, served a different purpose, checking the demonstration argument rather than shaping the framework during construction, and produced no traceable change to either the framework or the tool.

7.5 Quality Assurance of the Implementation

Testing is layered by what each layer can actually catch. At the time of writing the backend carries roughly 1,606 tests, the frontend roughly 305 unit tests, and there are 24 end-to-end test cases across 12 specifications; these counts are a snapshot of a codebase that continues to change and are reported as such rather than as a property of the system. The backend suite covers workflow logic, security boundaries and state machines, and depends on a set of automatically applied fixtures that remove categories of flaky-test risk by construction: coordination is forced off so no test depends on network reachability, project-management authentication is bypassed so route tests exercise business logic rather than third-party authentication, storage is pinned to local disk so no test can reach cloud storage, and the two context variables carrying per-request project isolation are reset between tests using the same mechanism production relies on. Frontend end-to-end specifications intercept at the browser network layer rather than by substituting modules in a test runner, because the requests being intercepted are the ones a real browser issues during real navigation.

Five independent jobs must pass in CI before any container image is built, and the deployment job captures the currently live image tags before changing anything, so that an automatic rollback target always exists. One of the five is a real-stack smoke test that boots the actual processes and asserts on live behaviour rather than on mocks, and it is deliberately narrow: it checks that an invalid token is rejected both with and without the header that selects a project management instance, because a prior regression allowed an invalid token to reach workspace data when validation anchored on the wrong value.

The defects worth reporting here are the ones with methodological significance rather than the ones that were merely difficult. Three concern the framework's own premise that generated output requires validation. A coverage badge reported the model's own claim about which scenarios it had covered, at face value; it is now reconciled server-side against the parsed specification, and a claimed scenario that does not match a real one is dropped rather than displayed. A section-parsing routine anchored on the writer's own headings, which the model also emits, and truncated a live project's technical specification part-way through, silently shortening the contract every later phase reads. And an automation step generated a test plan and then marked the story's quality gate as passed without any verification behind it, writing a fabricated result into the story index; fixing it exposed a second defect immediately downstream, where the deployment step assumed every story reaching it had genuinely passed.

Two others concern isolation, and they matter because isolation is what makes the specification store trustworthy. The identity anchor determining which tenant a request belongs to had to be corrected twice, once because a per-request value was allowed to override a validated one and once because a user-writable stored value was allowed into the chain at all. Separately, a listing endpoint belonging to

an integrated tool was found to return pages from projects other than the one requested, because that tool's own filtering does not strictly scope the result set; the fix re-filters every returned row rather than trusting the upstream response. The general lesson is the one the framework states in Section 6.6.2: a control is a measure whose absence makes an incorrect outcome undetectable, and each of these was detected only because something was checked against a reference rather than accepted as reported.

7.6 Limitations of the Implementation

The limitations of Apex as a system must be separated from the limitations of the framework as a methodology, because conflating them would let a defect in one be read as a refutation of the other.

Three are limitations of the system. The original single-writer constraint was a real ceiling rather than a theoretical one, and although the coordination layer described in Section 7.2 has lifted it in production, the fallback path remains the one most deployments would run. The end-to-end test fixtures are not statically checked against the real response contracts, so a new required field can be added to a response with the type checker and the unit suite both staying green; this was not hypothetical, and it is an open gap rather than a solved one. And Apex observes only what passes through it: it can see a story's specification chain, and, once a defect is reported through the project-management tool and linked to a story, a proxy signal that a deployment shipped a fault - but it has no way to observe a defect that is never reported there, and no way to attribute one to generated logic specifically.

Two are limitations that belong to the framework and were exposed by trying to build it, and both remain open even after the September 2026 work reported above. **AI Defect Escape Rate, at the observation point the framework's own definition presumes, is not instantiable by this class of system:** nothing in the framework says who owns production telemetry for generated logic specifically, and a tool sitting above a project-management board has no access to it regardless of who does. What is instantiable, and was added in September 2026, is the weaker proxy described above, which is a real signal but not the one the framework defines. **Spec-drift detection presumes a granularity the framework does not state**, namely that the specification is scoped per story rather than per project; the instantiation revealed the omission by failing on it. The functional specification meets that assumption and drift detection now works there; the technical specification, written as one project-wide contract, still does not, and remains open rather than closed. Both constructs are reported in Table 7.1 and both are carried into Chapter 9 as findings rather than as defects to be excused.

The two proxies differ in kind, and Table 7.1 names both as proxies rather than presenting either as the construct itself: Bolt Cycle Time was one briefly and by accident, corrected once the substitution was noticed, while AI Defect Escape Rate is one by design from the outset. A substitution left unnamed is indistinguishable from the thing it substitutes for.

8

Demonstration

Contents

8.1 Demonstration Design	52
8.2 Context	55
8.3 Application of the Framework	55
8.4 Observations	56
8.5 Summary	56

This chapter reports the Demonstration activity of the DSRM: the application of the framework, through its reference implementation, to a real software development project. The purpose of the demonstration is to show that the artefact can be used to address the research problem in a genuine setting, and to generate the evidence on which the evaluation in Chapter 9 rests.

8.1 Demonstration Design

This section is written before the demonstration is performed. Committing in advance to what will be observed, and against which criteria, is what separates a demonstration from an anecdote, and it is the only protection against the reasonable suspicion that the observations reported later were selected because they suited the artefact. Where a statement below turns out to be wrong once the work is done, it is corrected in Section 8.4 and the correction is marked, rather than the commitment being quietly revised.

Three settings, and what each can and cannot show

The framework is demonstrated in three settings rather than one, because they fail in different directions and no single one of them carries the claim alone.

That the instantiation counts as a demonstration at all is a framing adopted in response to the practitioner feedback reported in Section 7.4, not one this work started with, and the reframing is itself an instance of that formative evaluation. What it changes is what Apex is being asked to prove: not that it is a better way to obtain generated code, but that the governance constructs, phase gates enforced against specification content, Spec-Anchored Continuity, and classified loop-back, are implementable at all.

The first is **Apex itself**, and the claim needs stating precisely because it is easy to overstate. It is not that Apex's own development followed the framework's phases; it is that Apex is the framework rendered as executable software, so its construction demonstrates that the constructs are realisable rather than merely describable. In *DSRM* terms the instantiation is itself a solved instance of the problem: a working system in which every phase has a gate, every artefact resolves to a requirement, and no phase can be exited by generation alone answers the framework's own question, how AI participation across the lifecycle can be bounded, validated and traced, in a form that can be inspected rather than argued.

What that establishes is bounded: it is an existence proof rather than a trial. It shows the constructs can exist; it says nothing about whether they help anyone, whether a team would tolerate them, or whether they survive contact with work the designer did not anticipate. It is also reported retrospectively, since the system already exists, so it is explicitly not pre-registered and nothing in the record below applies to it. Its detail is not repeated here either: the construct-by-construct mapping in Section 7.3 is the demonstration's first result, and this chapter treats it as such rather than restating it.

The second is **Outfolio**, a portfolio and project-documentation platform for developers, separate from Apex and from this dissertation. It is the author's own project, so the researcher-role threat below applies to it in full, but it differs from the first setting in the way that matters: the framework is fixed before the work begins, and the software being built has nothing to do with software process, so the framework is applied to a domain it was not derived from. This is the setting in which the full lifecycle, Phases 1 to 6, is exercised end to end.

The third is the **partner organisation** already using Apex, described in Section 8.2. This is the only setting in which the framework is operated by people who did not design it, and so the only one capable of showing that the method is transferable rather than merely usable by its author. It also exercises the tool independence claimed in Section 6.2 in the only way that counts, since the organisation is migrating its project management to a different tool from the one Apex was first built against, and the framework's phases and gates have to survive that move unchanged even though the adapter beneath them does not. Its limitation is coverage: a working team engages the framework where it is useful to them, not exhaustively, so fewer phases will be exercised and the observations will be less complete. Following the consent commitment recorded in Section 9.2, the organisation is not named and identifying project detail is generalised.

Unit of analysis, scope and duration

The unit of analysis is **one Unit of Work carried from Strategic Alignment through to a deployed increment**, not a whole project and not an individual task. That level is chosen because it is the smallest unit at which every gate in Table 6.5 is exercised at least once, and because the framework's own claims are made about the passage of work through phases rather than about the output of any single phase.

Each setting contributes a defined number of such units, fixed here rather than after the fact: Outfolio contributes at least two, so that a second unit can be observed after the first has revealed how the framework behaves in that context, and the partner organisation contributes at least one. The instantiation setting has no unit count, for the reason given above. The demonstration runs until those units reach either a deployed increment or an abandonment recorded with its reason, whichever comes first; an abandoned unit is reported, since a framework that cannot carry work to completion in a real setting is a finding.

Prerequisite fieldwork, not yet done, blocking every section below: (1) finish Outfolio's second Unit of Work through to a deployed increment or a recorded abandonment - the first unit alone does not satisfy the commitment above; (2) contact the partner organisation, confirm at least one Unit of Work on their side of the PM-tool migration, and get it through to deployment or recorded abandonment. Neither Context, Application, Observations nor Summary below can be written until both exist as real, checkable evidence.

What is recorded, decided in advance

At every gate crossed, in every setting, the following is recorded: the artefact presented as evidence, the decision taken, the hat that took it, and, where the gate failed, the named phase the work was routed back to and what changed before it returned. Loop-backs are recorded as first-class events rather than as exceptions, for the reason stated in Section 6.4: a demonstration in which nothing was rejected and nothing looped back is evidence that the framework was not exercised, not evidence that it works.

Alongside the gate record, three further classes of observation are pre-committed. First, the governance metrics the framework defines in Section 6.6.4, to the extent the instantiation computes them, with any it does not compute, and any computed only as the proxies Table 7.1 names, reported as such rather than silently presented as the construct. Second, every point at which generated output was modified or rejected by a human, together with what was wrong with it, since the framework's central premise is that this happens and is caught. Third, every point at which the framework was found awkward, over-specified, or was bypassed in practice; these are recorded as they occur rather than reconstructed afterwards, and they are the most valuable input to Chapter 10.

Two commitments about reporting are made here so that they cannot be reconsidered later. Observations unfavourable to the artefact are reported with the same prominence as favourable ones. And the artefacts themselves, generated scenarios, specifications, task breakdowns and test plans, are reproduced rather than described, so that a reader can judge their quality instead of accepting a characterisation of it.

The researcher's role, and the threat it creates

The author designed the framework, built its instantiation, and is the participant in Outfolio. This is the most serious threat to the validity of this chapter and it is named here rather than deferred to Section 9.5.

The threat is not primarily that the author would misreport what happened. It is that a designer applying his own method is the person least likely to notice where it is awkward, because he already knows what it is for, supplies missing rationale from memory without registering that he is doing so, and reads an artefact as adequate against an intention rather than against what is written. The instantiation setting carries a related but distinct problem: because the framework and the system were designed together, a construct that proved inconvenient could be revised in the method rather than recorded as a difficulty, and Section 7.4 reports two occasions on which exactly that happened.

Three things mitigate it and none of them removes it. Pre-registering the record above, for the reason given at the head of this section, is what makes a later omission visible. The gate record is defined against artefacts that exist independently of the author's account of them, so a claim in this chapter can be checked against the specification store rather than taken on trust. And the third setting, for the reason given above, is the only structural rather than merely procedural mitigation available; where the first two settings and the third disagree, the third is given more weight, and any such disagreement is reported.

What remains unmitigated is stated plainly: for the instantiation and for Outfolio, this chapter reports what a designer observed while using his own design, and no arrangement within this study makes that equivalent to independent application.

8.2 Context

Describe the organisational setting, the team composition and the hats each participant wears, the existing tooling and process baseline, and the nature of the software being built. Establish the baseline concretely enough that a reader can judge what the framework changed.

Characterise the problems present in the setting before the framework was applied, mapping them back to the gaps identified in Chapter 4: inconsistent validation of generated code, upstream pressure on requirements as prototyping accelerates, testing complexity, and unclear responsibility boundaries over generated artefacts.

8.3 Application of the Framework

Walk each phase in turn. For each: what the AI produced, what the human accepted, modified or rejected, which gate was applied, and what the gate decided. Include real artefacts - generated scenarios, specifications, task breakdowns, test plans - as figures or listings rather than describing them.

Record the loop-backs explicitly. A demonstration in which nothing was rejected and nothing looped back is not evidence that the framework works; it is evidence that it was not exercised.

8.4 Observations

Report what was observed at the decision points defined in Section 8.1, including the governance metrics the framework defines. Report observations that were unfavourable to the artefact with the same prominence as those that were favourable.

Where the framework proved awkward, over-specified, or was silently bypassed in practice, record it. These observations are the most valuable input to the refinement discussion in Chapter 10.

8.5 Summary

Summarise what the demonstration establishes and, equally importantly, what it does not. Hand over to Chapter 9.

9

Evaluation

Contents

9.1 Evaluation Strategy	57
9.2 Participants	59
9.3 Instruments and Procedure	59
9.4 Results	64
9.5 Threats to Validity	64
9.6 Discussion	64

This chapter reports the Evaluation activity of the DSRM. Following Pries-Heje et al.'s distinction between *when* and *how* an artefact is evaluated [25], the evaluation carried out here is *ex post*, since it assesses the artefact after construction and on the basis of its actual use, and *naturalistic*, since it takes place in a real organisational setting with real users rather than in a laboratory.

9.1 Evaluation Strategy

Two artefacts were produced in Chapters 6 and 7: a framework and its instantiation. They are not evaluated by the same means, and conflating them would weaken both claims. Table 9.1 therefore states which instrument evaluates which artefact, and which research question it addresses.

State plainly that SUS and NASA-TLX measure the instantiation, not the methodology, and that the framework's own validity rests on the interviews, the analytical assessment, and the demonstration evidence. Positioning this limitation as a deliberate design of the evaluation is far stronger than conceding it under questioning.

Table 9.1: Evaluation instruments, target artefact, and research question addressed

Instrument	Artefact evaluated	DRQ	What it establishes
Demonstration in a real project	Framework and instantiation	DRQ3, DRQ4	That the artefact is applicable and produces the intended artefacts and decisions
System Usability Scale (SUS)	Instantiation (Apex)	DRQ4	Perceived usability of the supporting tool
NASA Task Load Index (NASA-TLX)	Instantiation (Apex)	DRQ4	Perceived cognitive workload imposed on the practitioner
Apex User Experience (UX) questionnaire (author-designed)	Instantiation (Apex), with two items scoped to the framework	DRQ4	Trust in AI-generated content, legibility of phase gates, and error surfacing, which SUS and NASA-TLX are not designed to reach
Practitioner interviews	Framework	DRQ1, DRQ2	Practicality, feasibility, and perceived value in real workflows
Agile and Scrum expert interviews	Framework	DRQ1, DRQ2	Alignment with, and departure from, established delivery methodology
Analytical assessment against criteria	Framework	DRQ1, DRQ2	Degree to which stated design criteria are satisfied

9.2 Participants

Report the number of participants, their roles, seniority, sector, and prior exposure to AI-assisted development. Note that SUS is generally considered to yield stable results from around twelve respondents, and state honestly what the achieved sample supports and what it does not.

Describe how participants were recruited: the channels used, how many were approached, and how many of those completed a session. State whether the sample was drawn from a single organisation or topped up from another population, since that determines what the external-validity claim can be.

No submission to an institutional ethics committee was required. The supervisor confirmed that for a study of this kind informed consent, together with anonymisation and a stated retention period, is sufficient at Instituto Superior Tecnico (IST), and that no standard institutional consent form exists that would supersede the one used here.

Every participant gave informed consent before any data was collected. The form, reproduced in Appendix B, states what the session involves, what is collected, that the artefact rather than the participant is under test, and that participation may be stopped at any point without a reason and without consequence. Consent is recorded as three separate items rather than one, so that agreeing to take part does not imply agreeing to be quoted or to be contacted for a follow-up interview; the three were tick boxes on the first page of an online form, completed before the task script was opened, so that no response was collected ahead of consent.

No session was observed and nothing was recorded: no screen capture, no audio and no video, of the task sessions or of the interviews, and the consent form contains no recording item because there was nothing to seek permission for. What was collected is the questionnaire responses, the timestamps at which each form was submitted, and, for the interviews, contemporaneous written notes taken by the researcher and written up in full immediately after each interview. The methodological cost of this choice, principally that a set of written notes is selective in a way a full transcript is not, is set out in Section 9.5 rather than left for the reader to infer.

Participants are identified only by a code of the form P1 to Pn. Names, employers, and identifying project details were not recorded with the responses, and quotations reproduced in this chapter are anonymised, with any detail that could identify a participant or their organisation removed or generalised. Only wording written down verbatim at the time is presented as a quotation; the remainder of the interview material is presented as paraphrase and is marked as such. The anonymised questionnaire responses and interview notes are retained, accessible only to the researcher, and are reproduced in Appendix B, so that the aggregate figures reported in Section 9.4 can be independently recomputed.

9.3 Instruments and Procedure

The usability study was administered entirely unmoderated. Each participant worked alone, at their own pace, from a fixed written task script, and submitted the questionnaires as online forms without the researcher present. A second, moderated arm of a small number of observed think aloud sessions was designed and deliberately dropped. Two reasons decided it. First, no session was to be recorded, and an observed session without a recording yields notes written by a facilitator who is simultaneously facilitating, which is the weakest form of the only evidence such an arm exists to produce. Second, and

more seriously, the author designed both artefacts under evaluation; an unmoderated design removes that conflict structurally rather than mitigating it, because there is no facilitator present to steer, hint, or react. What the decision costs, namely measured task completion, time on task, assist counts, and any observational account of where participants struggled, is stated in Section 9.5.

9.3.1 System Usability Scale

The SUS [52] is a ten-item instrument scored on a five-point scale of strength of agreement, administered after the participant has completed a defined set of tasks. Participants responding in Portuguese are given the validated European Portuguese version [53], with the only adaptation being the substitution of the system name for the generic referent used in the published items; participants responding in English are given the original wording. The two language versions are reported separately rather than pooled.

Report the scoring procedure and state the interpretation scale used, including the adjective ratings of Bangor et al. [54] and the curved grading scale of Sauro and Lewis [55], so that a single number is not left to speak for itself. Report the Usability and Learnability sub-scales separately [56]. State plainly that the score is not a percentage. State also the limits the Portuguese validation itself reports: construct validity was established, but inter-rater reliability was weak ($ICC = 0.36$) and agreement was 76.67 per cent, and the validation sample was drawn from the general community rather than from software practitioners.

9.3.2 NASA Task Load Index

The NASA-TLX [57] measures perceived workload across six subscales: mental demand, physical demand, temporal demand, performance, effort and frustration. This evaluation uses the unweighted Raw NASA-TLX variant [58], omitting the fifteen pairwise weighting comparisons. The instrument is administered once after each of the marked tasks rather than once at the end of the session, which is the instrument's intended per-task use and yields a workload profile per lifecycle phase rather than a single aggregate.

The Raw variant follows from that choice of administration: those fifteen comparisons would become ninety per participant under per-task administration, well beyond what the session can absorb. The weights also earn their cost primarily when workload is compared across experimental conditions, and this evaluation has no second condition against which they would do that work.

The variant is not, however, adopted only on grounds of feasibility. Reviewing five hundred and fifty studies twenty years after the instrument's publication, Hart records that removing the weighting is the most common modification made to the NASA-TLX, and that across the twenty-nine studies in which the two versions were directly compared the Raw variant was found to be more sensitive, less sensitive, or equally sensitive depending on the study, with no consistent advantage to weighting [58]. The subscale scores, which are this evaluation's primary result, are in any case identical under both versions, since the weighting affects only how they are combined into a single aggregate.

Two departures from the published instrument are recorded here rather than left to be noticed. Workload was collected on a linear scale from 0 to 10 rather than on the original twenty-interval line, and the responses were rescaled to the instrument's 0 to 100 range by multiplying by ten; no form tool suitable for an unmoderated study reproduces a twenty-interval line. The subscales were also reordered to place the reversed performance scale last rather than fourth, because a reversed scale encountered

in the middle of five forward ones, with no facilitator present to explain it, is the most plausible way a participant inverts a sixth of the data set. Neither change affects the unweighted Raw NASA-TLX computation. The first, however, reduces the resolution of the instrument: every response is a multiple of ten, so the resulting values are not directly comparable to studies using the original response scale, and small differences between cells of the workload matrix are not interpreted as meaningful.

Repeated administration carries its own cost, and it is a genuine trade rather than an oversight. Rating the same six dimensions six times invites anchoring, where later ratings drift towards earlier ones, and fatigue on the final tasks of the session. Neither is avoidable under per-task administration; both are carried into Section 9.5 and weighed there against the workload profile that per-task administration is what makes obtainable.

Define the task set both instruments are administered against, and keep it identical across participants. Place the full instruments in an appendix and reference them here.

9.3.3 Apex UX Questionnaire

An eighteen-item questionnaire was written for this evaluation to reach what SUS and Raw NASA-TLX are not designed to say: whether a participant trusted AI-generated content, whether a phase gate read as a guardrail or as obstruction, and whether reviewing generated work felt cheaper than writing it. Its items are grouped into five constructs: understanding where you are in the process, working with AI-generated content, control and reversibility, feedback and errors, and fit to real work. It is not a validated instrument and is not treated as one: it is reported item by item, with no composite score and no subscale means, since collapsing an unvalidated eighteen-item scale into a single figure is exactly the kind of claim an examiner would take apart. Administered last, after SUS, and reproduced in full in Appendix B.

Two items in the fifth construct, on whether the artefacts and the process fit real work, ask about the framework rather than the instantiation, and are analysed under the framework's evidence rather than Apex's, consistent with the separation stated in Table 9.1: keeping them on the same sheet is convenient for the participant, but splitting them out in analysis is what keeps the instrument from quietly undermining that separation.

State which item pair is expected to be the most diagnostic and why: the trust items (whether the participant trusted generated content enough to use it in a real project, against whether reviewing it took less effort than writing it) as a trust-calibration pair, and the item on understanding a deployment-gate refusal, cross-read against the corresponding NASA-TLX Frustration score for that task and its free-text response, since there is no observer-scored comprehension check in an unmoderated design.

9.3.4 Semi-Structured Interviews

Interviews were conducted with a population separate from the unmoderated task-based study reported in Section 9.2: practitioners and an Agile/Scrum expert recruited individually by the researcher rather than as a follow-up to that arm. This is a deliberate design choice rather than a gap. The interview guides in Appendix B ask about the framework's practicality, its adoption cost, and its alignment with established delivery methodology; these are questions a participant who has just spent an hour on nine fixed tasks is poorly placed to answer for their own team and organisation, and the Agile-expert guide

explicitly does not require the tool to have been used at all. The themes below were derived from the written notes described in Section 9.2: the codebook was developed from the first write-ups in each group and applied across the remainder, with codes added as they emerged. Session duration was not logged systematically and is not reported.

For the same anonymity reason stated in Section 9.2, interviewees are identified here only by a code: T1 to T5 for practitioners and E1 for the Agile/Scrum expert. These are kept deliberately separate from the P-codes used for the unmoderated study's participants, since the two populations are disjoint and neither did the other's protocol.

Five practitioners and one Agile expert had been interviewed at the time of writing, against Guide 1's target of five to eight practitioners and Guide 2's target of two to three experts; one practitioner, T3, was interviewed a second time after the framework had been substantially revised in response to the first round. Both groups therefore sit at or below the low end of their target, and recruitment continues past submission of this draft; the sample is reported at its current size rather than held back for a target that would postdate the deadline. What follows should be read at that scale, as a small number of attributable, individually rich sessions rather than a saturated qualitative sample, and the themes below name which interviewee raised which point precisely because the sample is too small for that attribution to be smoothed away by aggregation. Two of the five practitioners, T1 and T5, are not quoted below: their sessions took place earlier, during the tool's construction, and their material is reported instead in Section 7.4 as part of the framework's design history, which is where it did concrete work in changing the design; reporting it again here would be the double-counting Section 7.4 argues against. T2, T3 and T4 are the three whose interviews are reported as this section's evidence.

A - Practitioners. T2 offered the framework's central bet, that a locked specification should be the durable artefact and code should be regenerable from it, its clearest unprompted endorsement in this group: specifications, in his description, function like a library for project information and as the vehicle for guidelines, requirements, plans and tests kept together. The same practitioner separately named a real limit on that same idea, that it is difficult to account for every assumption a specification leaves implicit, and suggested grounding requirements by asking the user a structured series of questions to establish that foundation before generation begins.

T4 raised semantic versioning of the specification as a requirement rather than a suggestion, arguing that a human must always be able to consult and edit an artefact's history rather than trust it only as it currently stands. The same practitioner named, unprompted, the risk that a strictly linear phase sequence reintroduces waterfall-style rigidity, arguing for feedback loops rather than a one-directional pipeline. This second point is worth isolating because the Agile-expert guide in Appendix B was written anticipating exactly this criticism from methodology experts specifically; a practitioner raising it unprompted, without an expert being asked to probe for it, is stronger evidence that the concern is substantive than if it had only ever come from a session designed to elicit it.

T3's first interview raised two further points, feature-level rather than framework-level and each reported for completeness rather than as a developed theme: that AI-generated requirements need more detail specific to the project-management tool in use, Taiga in this case, and that the design should incorporate more UI factors to keep on-screen state clearly defined. T3's second interview produced the sharpest challenge the framework received in this study, and its content, together with the design

changes it is directly traceable into, is reported in full in Section 7.4 rather than here.

B - Agile/Scrum expert. The one expert interview obtained, following a live presentation of the framework and a demonstration of Apex, produced the clearest validation this study collected of the demonstration argument made independently in Chapter 8: that a tool instantiating and applying a framework's roles and phases makes the framework legible in a way a description on paper does not. The specific comparison volunteered was against the rigid, closed-role structures typical of methodology adopted inside large organisations, against which the framework's hats and its refusal to lock roles to named individuals read as a genuine point of difference; the Mob Elaboration ritual and the framework's governance metrics drew the same reaction, read as departures from what the expert's own organisation does today.

Two further observations from the same session are reported as open limitations rather than folded into that validation, because neither produced a design change. The first concerns Apex directly: navigating the tool mid-demonstration produced visible uncertainty about where a given action would lead next, and the expert named tooltips or contextual framing, and a tutorial, as what was missing. This is a first-contact observation from a single session rather than a defect evidenced the way Section 7.6 otherwise requires, and it is recorded as a gap rather than a measured finding. The second concerns the expert's own organisation rather than Apex: AI tool use there is centrally governed, restricted to a single permitted tool, and metered by token consumption, yet no role or specialist owns that governance, a gap large enough that the organisation is actively hiring against it. The expert raised, unprompted, whether AI participation in a lifecycle nets out as a cost or a benefit; the framework's governance metrics (Section 6.6.4) are positioned to answer exactly that question but do not yet operationalise it for token cost specifically, and the point is carried forward as a direction for future work in Chapter 10 rather than pursued further here.

C - Read across both groups. The strongest convergent signal is the linearity concern: raised unprompted by a practitioner using the tool, and anticipated independently in the design of the expert guide before any interview took place, it reaches this study from two directions that were never told to look for the same thing. The strongest divergence over the specification-as-source-of-truth construct spans two sections rather than sitting inside this one: T2's unprompted endorsement is reported above, and the sharpest doubt this study collected on the same construct comes from the same population but is reported in Section 7.4 rather than here, for the reason given there. Naming the split rather than letting the endorsement stand alone is deliberate: the favourable reading is not reported as though it were uncontested.

9.3.5 Analytical Assessment

State the success criteria in advance - clarity, completeness, alignment with SDLC phases, support for human-in-the-loop practice, and governance of AI-enabled activity - and the three-level ordinal scale used to assess each. Each rating must carry a short qualitative justification grounded in observed usage, artefacts, or interview evidence rather than in the author's judgement alone.

9.4 Results

Report **SUS** results: per-participant scores, mean, and dispersion. Given the expected sample size, report the distribution rather than the mean alone, and avoid inferential statistics the sample cannot support.

Report **NASA-TLX** results per subscale, not only the aggregate. The subscale profile is the informative part: a tool that raises mental demand while lowering frustration and effort tells a specific and reportable story about where the framework moves the practitioner's burden.

Report the Apex **UX** questionnaire results item by item as diverging bar charts, with the median per item, no composite score. Report the two framework-scoped items (process fit and effort-versus-value) separately from the sixteen instantiation-scoped items, and cross-read the deployment-gate comprehension item against the corresponding **NASA-TLX** Frustration score and free text.

Report the interview findings as themes, with representative anonymised quotations. Report disconfirming evidence explicitly.

Report the analytical assessment as a table of criteria, ratings, and justifications.

9.5 Threats to Validity

Address construct, internal, external, and conclusion validity. Name specifically: the author's dual role as designer and participant, and that the unmoderated design removes it during the sessions but not from the task set, the questionnaire wording, or the interviews, all of which the author authored and conducted; the small and non-random sample, recruited through the supervisor's professional connections, which makes it a convenience sample and bounds what the external-validity claim can be; the single organisational setting; the short observation window, which precludes any claim about long-term maintainability or organisational change; and the fact that the instruments measure perception rather than delivered software quality. For each threat, state the mitigation applied, or state plainly that none was possible.

Address the two threats that follow from the decision to observe and record nothing, and state them as consequences of a deliberate design rather than as oversights. First, there is no observational evidence at all: no measured completion rate, no time on task beyond an indicative form timestamp that cannot distinguish a hard task from an interruption, no assist or error counts, and no independent check on whether a participant understood the deployment refusal in task eight. This leaves standing, rather than partly answering, the threat that the instruments measure perception rather than delivered software quality. Second, the interview analysis rests on written notes rather than transcripts, and note taking is selective in a way transcription is not, with the selection made by the person who designed the artefact.

9.6 Discussion

Synthesise across instruments. Where they agree, say so; where they disagree - for instance a favourable usability score alongside interview scepticism about the process overhead - treat the disagreement as a finding rather than smoothing it over, and read it against the threats to validity stated above.

9.6.1 Answering the Research Questions

Answer DRQ1, DRQ2, DRQ3 and DRQ4 from Chapter 1 in turn, each grounded explicitly in the evidence reported above (results, demonstration, interviews) rather than merely asserted. DRQ1 and DRQ2 concern the framework and are answered primarily from the practitioner and Agile-expert interviews and the analytical assessment; DRQ3 is answered from the construct-to-mechanism mapping in Section 7.3 read together with the implementation limitations, reporting which constructs were realised in full, which were realised as proxies, and which resisted implementation, since the existence of Apex alone establishes only that something was built; DRQ4 is answered from SUS, NASA-TLX, the Apex UX questionnaire, and the demonstration. This closes the loop the introduction opened: each research question is answered here with evidence, not merely restated in the conclusion.

10

Conclusion

Contents

10.1 Summary and Contributions	66
10.2 Communication	66
10.3 Limitations	67
10.4 Future Work	67
Declaration of Generative AI and AI-Assisted Technologies in the Writing Process	68

10.1 Summary and Contributions

Restate the problem, the artefacts produced, and what the evaluation established. Summarise, rather than re-derive, how each research question from Chapter 1 was answered; the evidence-grounded answers themselves belong in Section 9.6.1. Keep this to what the evidence actually supports.

10.2 Communication

This section addresses the Communication activity of the DSRM.

Report the dissemination plan and its current status: the scientific article reporting the SLR and its target venue; the report presenting the framework; and this dissertation, formally presented and defended before an academic examination committee. Update the status of each submission before final delivery.

10.3 Limitations

State the limitations of the research honestly: the framework's core constructs are an original synthesis rather than being drawn wholesale from a single validated source; several grounding sources are recent, vendor-reported, or not yet peer-reviewed, and should be cited as such; the governance-standards vocabulary is used as shared language and not as a compliance claim; and the evaluation covers early adoption in a single setting, so it cannot speak to sustained organisational change.

10.4 Future Work

Derive future work from the evaluation findings rather than listing generic extensions. Candidates: application across multiple organisations and team sizes to test generalisation; longitudinal measurement of defect escape and maintainability, which the current evaluation window cannot reach; support for project management backends beyond the one currently integrated; replacing proxy metrics with direct measurement where production telemetry becomes available; and operationalising the framework's governance metrics for AI token cost specifically, raised by the Agile-expert interview in Section 9.3.4 as a real, currently unowned gap in at least one organisation adopting AI tooling.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work, the author used Claude Code, with the Sonnet 5, Opus 5 and Fable 5 models, together with Google Gemini and Gemini Notebook, to improve the wording and clarity of the manuscript and to support the organisation and review of source material. After using these tools, the author reviewed and edited the text as needed and takes full responsibility for the content of the publication. The final manuscript reflects the author's own analysis, results, and conclusions.

This declaration concerns the use of these technologies in the writing process only. The use of generative AI as the object of study of this research, and within the artefacts it produces, is described in Chapter 6 and Chapter 7.

Bibliography

- [1] J. H. Moore and N. Tatonetti, “Vibe coding: a new paradigm for biomedical software development,” *BioData Mining*, vol. 18, no. 1, pp. 1–3, 2025.
- [2] A. Zamfiroiu, C.-B. Păștinică, C. Crăciun, C. Ciutacu, and T. Luca, “Exploring Conversational Programming through the CHOP Paradigm and Artificial Intelligence,” *Informatica Economica*, vol. 29, no. 2, pp. 5–17, 2025.
- [3] I. Ahmed, A. Aleti, H. Cai, A. Chatzigeorgiou, P. He, X. Hu, M. Pezzè, D. Poshyvanyk, and X. Xia, “Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–27, 2025.
- [4] S. Zhang, Z. Xing, R. Guo, F. Xu, L. Chen, Z. Zhang, X. Zhang, Z. Feng, and Z. Zhuang, “Empowering Agile-Based Generative Software Development through Human-AI Teamwork,” *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 6, pp. 1–46, 2025.
- [5] W. Hou and Z. Ji, “Comparing Large Language Models and Human Programmers for Generating Programming Code,” *Advanced Science*, vol. 12, no. 8, pp. 1–12, 2025.
- [6] Stack Overflow, “2025 Developer Survey: AI,” Annual practitioner survey, industry-reported, not peer-reviewed, 2025. [Online]. Available: <https://survey.stackoverflow.co/2025/ai/>
- [7] DORA and Google Cloud, “State of AI-assisted Software Development 2025,” Industry research report, not peer-reviewed, 2025. [Online]. Available: <https://dora.dev/dora-report-2025/>
- [8] S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li, “Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks,” arXiv:2512.03262, preprint, not yet peer-reviewed, 2025. [Online]. Available: <https://arxiv.org/abs/2512.03262>
- [9] Veracode, “2025 GenAI Code Security Report: Assessing the Security of Using LLMs for Coding,” Industry report, not peer-reviewed. Assessment of over 100 large language models across Java, Python, C# and JavaScript, July 2025. [Online]. Available: https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf
- [10] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” arXiv:2302.06590, preprint, not published in a peer-reviewed venue, 2023. [Online]. Available: <https://arxiv.org/abs/2302.06590>

- [11] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity," arXiv:2507.09089, METR, preprint, not yet peer-reviewed, 2025. [Online]. Available: <https://arxiv.org/abs/2507.09089>
- [12] Faros AI, "The AI Productivity Paradox Report 2025," Industry telemetry report, not peer-reviewed. Sample of over 10,000 developers across 1,255 teams, July 2025. [Online]. Available: <https://www.faros.ai/blog/ai-software-engineering>
- [13] L. Banh, F. Holldack, and G. Strobel, "Copiloting the future: How generative AI transforms Software Engineering," *Information and Software Technology*, vol. 183, 2025.
- [14] M. Farhanna, N. Paramitha, M. Susi, H. Surya, A. Yanti, P. Dea, and S. Deni, "Exploring the Use of Generative AI in Software Development: A Preliminary Study," in *E3S Web of Conferences*, vol. 645, 2025, p. 04002.
- [15] A. Hemmat, M. Sharbaf, S. Kolahehdouz-Rahimi, K. Lano, and S. Y. Tehrani, "Research directions for using LLM in software requirement engineering: a systematic review," *Frontiers in Computer Science*, vol. 7, 2025.
- [16] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin, "The Current Challenges of Software Engineering in the Era of Large Language Models," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–30, 2025.
- [17] H. A. Al-Hashimi, R. A. Khan, H. S. Alwageed, A. M. Algarni, S. Ayouni, and A. O. Almagrabi, "Exploring the role of generative AI in enhancing cybersecurity in software development life cycle," *Array*, vol. 28, p. 100509, 2025.
- [18] X. Bai, S. Huang, C. Wei, and R. Wang, "Collaboration between intelligent agents and large language models: A novel approach for enhancing code generation capability," *Expert Systems With Applications*, vol. 269, 2025.
- [19] M. Basha and G. Rodríguez-Pérez, "Trust, transparency, and adoption in generative AI for software engineering: insights from twitter discourse," *Information and Software Technology*, 2025.
- [20] X. Chen, C. Gao, C. Chen, G. Zhang, and Y. Liu, "An Empirical Study on Challenges for LLM Application Developers," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 7, pp. 1–37, 2025.
- [21] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-Collaboration Code Generation via ChatGPT," *ACM Transactions on Software Engineering & Methodology*, vol. 33, no. 7, pp. 1–38, 2024.
- [22] U. K. Durrani, M. Akpınar, M. F. Adak, A. T. Kabakus, M. M. Ozturk, and M. Saleh, "A Decade of Progress: A Systematic Literature Review on the Integration of AI in Software Engineering Phases and Activities (2013-2023)," *IEEE Access*, vol. 12, pp. 171 185–171 204, 2024.
- [23] Y. Han and C. Lyu, "Multi-stage guided code generation for Large Language Models," *Engineering Applications of Artificial Intelligence*, vol. 139, 2025.

- [24] M. A. Haque, "LLMs: A game-changer for software engineers?" *BenchCouncil Transactions on Benchmarks, Standards & Evaluations*, vol. 5, no. 1, pp. 30–41, 2025.
- [25] J. R. V. Jan Pries-Heje, R. Baskerville, "Strategies for Design Science Research Evaluation," *European Conference on Information Systems (ECIS)*, no. 1, pp. 1–13, 2008.
- [26] S. K. Jabrw and Q. I. Sarhan, "A Systematic Survey on Large Language Models for Code Generation," *ARO-The Scientific Journal of Koya University*, vol. 13, no. 2, 2025.
- [27] V. V. Jensen, A. Alami, A. R. Bruun, and J. S. Persson, "Managing expectations towards AI tools for software development: a multiple-case study," *Information Systems & e-Business Management*, pp. 1–33, 2025.
- [28] U. Karlovs-Karlovskis, "Generative Artificial Intelligence Use in Optimising Software Engineering Process: A Systematic Literature Review," *Applied Computer Systems*, vol. 29, no. 1, pp. 68–77, 2024.
- [29] D.-K. Kim and H. Ming, "Assessing output reliability and similarity of large language models in software development: A comparative case study approach," *Information and Software Technology*, vol. 185, 2025.
- [30] D.-K. Kim, "Improving Software Development Traceability with Structured Prompting," *Journal of Computer Information Systems*, pp. 1–21, 2025.
- [31] J. Li, G. Li, Y. Li, and Z. Jin, "Structured Chain-of-Thought Prompting for Code Generation," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 2, pp. 1–23, 2025.
- [32] V. S. Pendyala and N. B. Thakur, "Performance and interpretability analysis of code generation large language models," *Neurocomputing*, vol. 656, 2025.
- [33] K. Qiu, N. Puccinelli, M. Ciniselli, and L. Di Grazia, "From Today's Code to Tomorrow's Symphony: The AI Transformation of Developer's Routine by 2030," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–17, 2025.
- [34] T. Ramos, A. Dean, and D. McGregor, "AI-Augmented Software Engineering: Revolutionizing or Challenging Software Quality and Testing?" *Journal of Software: Evolution & Process*, vol. 37, no. 2, pp. 1–7, 2025.
- [35] V. Terragni, A. Vella, P. Roop, and K. Blincoe, "The Future of AI-Driven Software Engineering," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–20, 2025.
- [36] A. Tkachuk, "Determining the Capabilities of Generative Artificial Intelligence Tools to Increase the Efficiency of Refactoring Process," *Technology Audit & Production Reserves*, vol. 3, no. 2(83), pp. 6–11, 2025.
- [37] Q. Wang, J. Wang, M. Li, Y. Wang, and Z. Liu, "A Roadmap for Software Testing in Open-Collaborative and AI-Powered Era," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 5, pp. 1–17, 2025.

- [38] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, "Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models," *ACM Transactions on Software Engineering & Methodology*, vol. 34, no. 7, pp. 1–25, 2025.
- [39] S. H. Yoo and H. J. Kim, "Security Analysis of Automated Code Generation: Structural Vulnerabilities in AI-Generated Code," *Tehnički Glasnik*, vol. 19, no. 4, pp. 560–574, 2025.
- [40] Thoughtworks, "Spec-driven development," Technology Radar, Vol. 33, November 2025. [Online]. Available: <https://www.thoughtworks.com/en-us/radar/techniques/spec-driven-development>
- [41] S. E. Farrag, "The Productivity-Reliability Paradox: Specification-Driven Governance for AI-Augmented Software Development," arXiv:2605.01160, preprint, not yet peer-reviewed, 2026. [Online]. Available: <https://arxiv.org/abs/2605.01160>
- [42] Amazon Web Services, "AI-Driven Development Lifecycle (AI-DLC)," Vendor methodology announced at AWS DevSphere 2025. Reported productivity figures are vendor-reported and not independently verified, 2025. [Online]. Available: <https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/>
- [43] H. Kniberg and A. Ivarsson, "Scaling Agile @ Spotify with Tribes, Squads, Chapters & Guilds," Whitepaper, industry-reported, not peer-reviewed. A description of Spotify's organisation at the time of writing, not a prescriptive method, November 2012. [Online]. Available: <https://blog.crisp.se/2012/11/14/henrikkniberg/scaling-agile-at-spotify>
- [44] Leading Complexity, "Summary of Henrik Kniberg's session, about his thoughts on the Spotify Model, in the Leading Complexity programme 2022," Crisp's Blog. Third-party published summary of Henrik Kniberg's session in the Leading Complexity programme 2022, not written by Kniberg himself, February 2023. [Online]. Available: <https://blog.crisp.se/2023/02/06/leadingcomplexity/leading-complexity-series-summary-1-henrik-knibergs-thoughts-on-the-spotify-model>
- [45] J. Lee, "Failed #SquadGoals," Practitioner account by a former Spotify employee, industry-reported, not peer-reviewed, April 2020. [Online]. Available: <https://www.jeremiahlee.com/posts/failed-squad-goals/>
- [46] M. Skelton and M. Pais, *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. Portland, OR: IT Revolution Press, 2019.
- [47] C. Bryar and B. Carr, *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. New York, NY: St. Martin's Press, 2021.
- [48] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," National Institute of Standards and Technology, U.S. Department of Commerce, Technical Report NIST AI 100-1, January 2023. [Online]. Available: <https://doi.org/10.6028/NIST.AI.100-1>
- [49] ISO/IEC, "Information technology - Artificial intelligence - Management system," International Organization for Standardization/International Electrotechnical Commission, International Standard ISO/IEC 42001:2023, December 2023.

- [50] S. Siddeeq, M. Abbasi, J. Rasku, Z. Zhang, F. Christophe, T. Mikkonen, and P. Abrahamsson, "Epic-Organized vs. Requirement-Aligned Gherkin: An Empirical Evaluation of LLM-Based Acceptance Criteria Generation," Accepted at the International Conference on Software Engineering of Emerging Technologies (SEET 2026), Springer Nature. arXiv:2607.01980, 2026. [Online]. Available: <https://arxiv.org/abs/2607.01980>
- [51] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler, "The SPACE of Developer Productivity: There's More to it than You Think," *ACM Queue*, vol. 19, no. 1, pp. 20–48, 2021.
- [52] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. Taylor and Francis, 1996, pp. 189–194.
- [53] A. I. Martins, A. F. Rosa, A. Queirós, A. Silva, and N. P. Rocha, "European Portuguese validation of the System Usability Scale (SUS)," *Procedia Computer Science*, vol. 67, pp. 293–300, 2015. [Online]. Available: <https://doi.org/10.1016/j.procs.2015.09.273>
- [54] A. Bangor, P. Kortum, and J. Miller, "Determining what individual SUS scores mean: Adding an adjective rating scale," *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.
- [55] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Morgan Kaufmann, 2016.
- [56] J. R. Lewis and J. Sauro, "The factor structure of the System Usability Scale," in *Human Centered Design, HCII 2009, Lecture Notes in Computer Science*, vol. 5619, 2009, pp. 94–103.
- [57] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research," in *Human Mental Workload*, P. A. Hancock and N. Meshkati, Eds. North-Holland, 1988, pp. 139–183.
- [58] S. G. Hart, "NASA-Task Load Index (NASA-TLX); 20 years later," in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 50, 2006, pp. 904–908.



Extended Results for the Systematic Literature Review

This appendix holds the evidence tables underlying the review reported in Chapter 4. They are reproduced here rather than in the chapter itself so that the chapter can state what the review found without the reader having to read four tabulations to reach it, while the tabulations remain available in full for anyone auditing the synthesis. Kitchenham's guidelines require the review to be reconstructible from the protocol together with this material.

A.1 Distribution of the retained corpus by venue

The aggregate distribution is stated in Chapter 4; Table A.5 below gives the venue of every retained study individually, so that the concentration reported there, ten articles in ACM Transactions on Software Engineering & Methodology, three in Information and Software Technology, and the remaining eighteen spread one each across eighteen distinct venues, can be checked study by study.

A.2 Stakeholder opinions reported in the literature

The fourth research question is answered in Chapter 4 from the three tables below, which separate the perspectives by the group holding them: the public and researchers, developers and practitioners, and organisations and management. The separation is retained because the three groups disagree, and the disagreement is itself a finding.

Table A.1: Public / Researchers Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
General Trust / Perception	People's perception of systems powered by artificial intelligence as a mixture of "software and sorcery." [3]	6	[2, 3, 18, 19, 32, 34]
Misinformation Risk	"Users highlight its potential to misguide or hallucinate to students." [19]	4	[13, 19, 22, 26]
Bias and Impartiality Concerns	Users express dissatisfaction over perceived "leftist bias and propaganda in ChatGPT." [19]	3	[3, 19, 34]
Output Diversity / Similarity	"57% similarity overall with 43% variability, presenting that each model has its unique characteristics." [29]	2	[4, 29]
Legal, Copyright, and Licensing Issues	"Questions also surround the copyrightability/ownership of generated code [...]" [3]	2	[3, 19]
Evaluation Urgency	Urgent need to "establish an effective, objective, and complete evaluation system" for code-generation LLMs. [3]	2	[3, 32]
Job Automation	"Many users fear that AI will eventually replace human developers [...]" [19]	1	[19]

Table A.2: Developers / Practitioners Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
Productivity and Efficiency Gains	"The knowledge and reasoning capabilities of LLMs have the potential to result in significant efficiency gains." [13]	6	[2, 13, 19, 23, 24, 35]
Reliability Issues / Hallucinations	"LLMs sometimes produce highly convincing content that is factually wrong [...] the 'hallucination phenomenon'." [18]	6	[13, 18, 19, 22, 26, 32]
Output Variability and Trust Decline	"I get a different code each time [...] I lose a bit of confidence in the work." [13]	5	[13, 19, 32, 34, 35]
Quality of Generated Code	"Several quality issues: undefined variables, insecure comments, and code requiring significant revision." [26]	4	[4, 26, 27, 35]
Augmentation of Human Capabilities	"LLMs are not merely a product of overhyped marketing; they represent a profound shift in how software is engineered. They should be seen as powerful tools that augment human capabilities rather than replace them." [24]	3	[19, 24, 35]
Learning and Skill Improvement	LLMs "[...] directly support learning processes through interaction and conceptual clarification." [2]	3	[2, 16, 24]
Knowledge Acquisition / Search Replacement	"LLMs present data in a concise and contextually relevant manner." [13]	2	[2, 13]

Continued on next page

Opinion Category	Description	#	References
Controllability	Participants praised its "controllability [...] allowing modification and feature adjustment after generation." [4]	2	[4, 19]
Risk of Over-reliance and Cognitive Decline	"There is almost the danger that you stop thinking for yourself [...] people will tend to become lazy." [13]	2	[13, 19]
Investment of time with prompts	"You spend ages trying to cut the prompt so that it comes out right [...]" [13]	2	[13, 19]

Table A.3: Organizations / Management Opinions on AI and LLMs in Software Development

Opinion Category	Description	#	References
Need for Human Oversight	LLM outputs "still necessitate extensive human review before implementation." [29]	7	[4, 17, 19, 29, 30, 33, 35]
Data Security, Privacy, and Corporate Bans	"Due to concerns regarding data privacy, when considering real-world practice, most software organizations tend to avoid using commercial LLMs and would prefer to adopt open source ones with training or fine-tuning using organization-specific data." [37]	6	[13, 15, 19, 35, 37, 39]
Operational Costs and Resource Constraints	Organizations must "consider the current limitations in terms of computational power or pay close attention to energy consumption," leading them to "tend to fine-tune medium-sized models". [37]	3	[13, 15, 37]
Strategic Goal for Developer Satisfaction	"We have to be able to compete on efficiency and job satisfaction [...] It is difficult to find software developers." [27]	2	[24, 27]
Expectation Management Requirement	"SDOs must carefully manage their expectations towards AI tools." [27]	2	[19, 27]
Strategic Competitiveness	The "early adoption of LLMs in software engineering is crucial to stay competitive in this rapidly evolving landscape." [24]	1	[24]

A.3 Industry sectors adopting AI and LLMs

The evidence behind the fifth research question, answered in Chapter 4.

Table A.4: Industry sectors adopting AI and LLMs

Industry Sector	Description	#	References
Software Engineering, Software Development & Consulting Services	Software Development and Consulting is mentioned a lot, since they are the main sectors where AI and LLMs are being utilized in many different ways, such as AIOps, Cloud Management, Requirements Engineering, Software Testing, Quality Assurance, Code Generation and Development, Cybersecurity, Vulnerability Management, Refactoring, Code Optimization, Specialized Hardware, Scientific Computing and Emerging Technologies (like Blockchain/Crypto).	25	[2–4, 13–20, 22–24, 26–29, 32, 33, 35–39]
Finance, Banking, and Regulated Industries (Including Insurance)	Specialized LLMs can understand the "regulatory requirements, security needs, and performance constraints" especially in blockchain, cryptocurrency, smart contracts, and regulated environments. [24, 27, 29]	8	[4, 13, 17, 19, 24, 27, 29, 34]
Transportation and Safety-Critical Domains	The lack of transparency of LLMs is problematic, particularly in "safety-critical applications like healthcare, finance, or aerospace, where understanding the reasoning behind code is essential for ensuring security and correctness". [3]	7	[3, 15, 19, 24, 34, 37, 39]
Healthcare, Medical, and Biomedical Research	AI "accelerates the coding process by converting natural language or abstract research intent (a vibe) into functioning software modules, dramatically shortening development cycles in biomedical research and clinical application." [1].	5	[1, 4, 5, 18, 19]
Education and Training	LLMs "directly support learning processes through interaction and conceptual clarification". Students should be taught "how to specify AI, including functional, non- functional, and 'happy and unhappy path scenarios'" and learn "prompt engineering workarounds" and to "research and fact- check AI responses". [34]	5	[2, 4, 5, 15, 34]
Legal Reasoning and Regulatory Compliance	LLMs demonstrate success across multiple fields, such as education, healthcare, and legal reasoning. Specifically, prompt engineering is used to "tailor LLMs for the US legal system, enabling the model to identify relevant cases and predict judicial outcomes with greater precision". [26]	4	[17, 18, 26, 34]
Digital Ecosystems (E-Commerce and Services)	Case studies involve the development of systems such as a food order and delivery system (FODS) and a smart wallet system (SWS). LLMs are applied in contexts like Online Retailers and the service industry. [4, 29]	3	[4, 17, 29]

A.4 Final set of studies retained by the review

Table A.5 lists the thirty-one studies retained after the filtering process described in Chapter 4, so that the review can be reconstructed from this appendix together with the protocol reported there, as Kitchenham's guidelines require. The reference resolves to the full bibliographic entry; the last column states

which of the five research questions listed in Table 4.1 each study was drawn on to answer, following the same assignment used to build Tables 4.4 to 4.6 and A.1 to A.4. A study contributing to several research questions appears once here, with every question it answers listed, rather than once per table.

Table A.5: Studies retained by the systematic literature review

Reference	Venue	Year	RQs answered
[3]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5
[17]	Array	2025	1, 2, 4, 5
[18]	Expert Systems With Applications	2025	1, 2, 3, 4, 5
[13]	Information and Software Technology	2025	1, 2, 3, 4, 5
[19]	Information and Software Technology	2025	1, 2, 4, 5
[20]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 5
[21]	ACM Transactions on Software Engineering & Methodology	2024	1, 3
[22]	IEEE Access	2024	1, 2, 3, 4, 5
[14]	E3S Web of Conferences	2025	1, 2, 5
[16]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5
[23]	Engineering Applications of Artificial Intelligence	2025	1, 2, 3, 4, 5
[24]	BenchCouncil Transactions on Benchmarks, Standards & Evaluations	2025	1, 2, 3, 4, 5
[15]	Frontiers in Computer Science	2025	1, 2, 3, 4, 5
[5]	Advanced Science	2025	2, 5
[26]	ARO-The Scientific Journal of Koya University	2025	1, 2, 3, 4, 5
[27]	Information Systems & e-Business Management	2025	1, 2, 4, 5
[28]	Applied Computer Systems	2024	1, 2, 5
[29]	Information and Software Technology	2025	1, 2, 3, 4, 5
[30]	Journal of Computer Information Systems	2025	1, 2, 3, 4
[31]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3
[1]	BioData Mining	2025	1, 3, 5
[32]	Neurocomputing	2025	2, 3, 4, 5
[33]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5
[34]	Journal of Software: Evolution & Process	2025	1, 2, 3, 4, 5
[35]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5
[36]	Technology Audit & Production Reserves	2025	1, 2, 5
[37]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5

Continued on next page

Reference	Venue	Year	RQs answered
[38]	ACM Transactions on Software Engineering & Methodology	2025	2, 3, 5
[39]	Tehnički Glasnik	2025	1, 2, 4, 5
[2]	Informatica Economica	2025	1, 2, 3, 4, 5
[4]	ACM Transactions on Software Engineering & Methodology	2025	1, 2, 3, 4, 5

The search string and the search method are already stated in full in Table 4.2.

B

Evaluation Instruments

This appendix reproduces every instrument administered during the evaluation described in Chapter 9, in the order a participant encountered them: the consent and demographics form, the workload questionnaire, the usability questionnaire, the author-designed UX questionnaire, and the two interview guides used in the follow-up sessions. Participants chose English or Portuguese for the session (item G6 on the demographics form); every instrument was offered in both languages, and each is reproduced here in English, the language of this dissertation, with the source of the Portuguese wording stated alongside it. Scoring procedures, interpretation scales and results are reported in Chapter 9, not repeated here; this appendix is the instrument as administered, not its analysis.

B.1 Informed consent and demographics

Completed first, before any task or questionnaire; the ethics position it rests on is stated in Section 9.2. There is no recording-consent item, because no session and no interview was recorded.

Study: Evaluation of Apex, a tool supporting a process framework for human-AI collaboration in software development. **Researcher:** Tomás dos Santos Taborda, MEIC-T, Instituto Superior Técnico, Universidade de Lisboa. **Supervisors:** Prof. Miguel Mira da Silva, Hugo de Sousa.

What this involves. You will use a software tool for about 60 minutes to complete nine predefined tasks, working on your own and at your own pace. After six of those tasks you will fill in a short workload questionnaire, which takes about a minute each time. At the end of the session you will fill in two further short questionnaires, taking about 20 minutes in total. You may optionally be invited to a separate interview of 30 to 45 minutes.

What is being tested. The tool and the process behind it are being tested, not you. There is no way for you to perform badly. Difficulties you have are findings about the tool and are the most useful thing you can give us.

What is recorded. Only your answers to the questionnaires, and the times at which you submit each form. Nobody watches the session. Your screen is not recorded, your audio is not recorded, and no video is taken at any

point. If you take part in the follow-up interview, that is not recorded either; the researcher takes written notes.

Anonymity. You are identified only by a code such as P3. Your name, your employer and any project details you mention are not recorded in the results. Quotations used in the dissertation are anonymised, and any detail that could identify you or your organisation is removed or generalised.

Data handling. There are no recordings to store or delete, because none are made. Anonymised questionnaire responses and anonymised interview notes are stored, accessible only to the researcher, and are kept so that reported results can be independently recomputed. They may be reproduced in an appendix of the dissertation in anonymised form.

Voluntary participation. Participation is voluntary. You may take a break at any time, skip any task or question, or stop and withdraw entirely without giving a reason and without any consequence. If you withdraw, tell the researcher whether your data so far should be deleted or kept.

Risks. There are no anticipated risks beyond mild frustration with software.

I have read and understood the above, and I agree to take part.

- ☐ I confirm I have read and agree to the above. (required)
- ☐ I consent to anonymised quotations being used in the dissertation.
- ☐ I agree to be contacted about a follow-up interview.

Participant code (from your card): _____

Demographics

Filled in immediately after consent, on the same form, in the language the participant chose. The participant code is the join key back to the workload, usability and UX forms.

Item	Question	Options
G1	Years of professional software development experience	under 1 / 1-3 / 4-7 / 8-15 / over 15 / student
G2	Current role	developer / tester or QA / product owner / designer / engineering manager / student / other
G3	Frequency of use of AI coding assistants	never / tried once or twice / monthly / weekly / daily
G4	Familiarity with Gherkin or BDD	none / have seen it / use it regularly
G5	Familiarity with the project management tool used in the session	none / some / use it regularly
G6	Preferred language for the session	English / Português

G3 is expected to be the strongest confound in the results and is reported alongside them rather than left implicit.

B.2 Raw NASA Task Load Index

The Raw NASA-TLX [57, 58] was administered once after each of the six marked tasks (script tasks 3, 4, 6, 7, 8 and 9), identified by a required first question naming the task and a second question for the

participant code. Workload was collected on a linear scale from 0 to 10 and rescaled by multiplying by ten, and the subscales were presented in the order below, placing the reversed Performance scale last rather than fourth; both departures, and the reason for each, are stated in Section 9.3.

Subscale	Description	Scale
Mental Demand	How much mental and perceptual activity was required? Was it easy or demanding, simple or complex, forgiving or exacting?	Very Low - Very High
Physical Demand	How much physical activity was required? Was the task easy or demanding, slow or brisk, restful or laborious?	Very Low - Very High
Temporal Demand	How much time pressure did you feel due to the pace of the task? Slow and leisurely or rapid and frantic?	Very Low - Very High
Effort	How hard did you have to work, mentally and physically, to accomplish your level of performance?	Very Low - Very High
Frustration Level	How insecure, discouraged, irritated, stressed and annoyed, versus secure, gratified, content and relaxed, did you feel?	Very Low - Very High
Performance (reversed anchors)	How successful were you in accomplishing this task's goals?	Perfect - Failure

Administered in Portuguese to participants selecting that language on G6, using the direct translations in docs/thesis/evaluation/instrument-nasa-tlx.md: same six subscales, same order, same reversed Performance anchors, differing only in wording.

B.3 System Usability Scale

The SUS [52] was administered after Raw NASA-TLX, self-completed and unwatched, scale 1 = Strongly disagree ... 5 = Strongly agree. The only adaptation from the standard wording is substituting Apex for the system. Participants responding in Portuguese were given the European Portuguese version validated by Martins et al. [53], whose only change from the published items is the same substitution of the system name for the generic referent produto.

#	Item
1	I think that I would like to use Apex frequently.
2	I found Apex unnecessarily complex.
3	I thought Apex was easy to use.
4	I think that I would need the support of a technical person to be able to use Apex.
5	I found the various functions in Apex were well integrated.
6	I thought there was too much inconsistency in Apex.
7	I would imagine that most people would learn to use Apex very quickly.
8	I found Apex very cumbersome to use.
9	I felt very confident using Apex.
10	I needed to learn a lot of things before I could get going with Apex.

Administered in Portuguese to participants selecting that language on G6, using Martins et al.'s published items [53] verbatim, with the same system-name substitution. That validation reports established

construct validity against the PSSUQ ($r = 0.70$) but weak inter-rater reliability ($ICC = 0.36$, agreement 76.67%), obtained on a general-community sample rather than a software-practitioner one; both points are carried into the reporting in Chapter 9 rather than left implicit here.

B.4 Apex UX Questionnaire

Not a validated instrument; its rationale and its five constructs are stated in Section 9.3.3. Administered last, scale 1 = Strongly disagree ... 5 = Strongly agree plus an explicit N/A - did not use this, reported item by item with no composite score. Items E2 and E3 evaluate the framework rather than the instantiation and are analysed under the framework's evidence in Chapter 9, consistent with Table 9.1.

A. Understanding where you are

- A1 At any moment, I knew which phase of the process I was in.
 - A2 I understood what the tool expected me to do next.
 - A3 I could tell the difference between work the tool had finished and work it had not.
 - A4 When the tool refused to let me continue, I understood why.
-

B. Working with AI-generated content

- B1 I could tell which parts of the content were generated by AI and which were mine.
 - B2 Reviewing the AI's output took less effort than writing it myself would have.
 - B3 I felt able to reject or change the AI's output when I disagreed with it.
 - B4 I trusted the generated content enough to use it in a real project.
 - B5 I could see what information the AI had been given as a basis for its output.
-

C. Control and reversibility

- C1 I felt in control of what the tool did on my behalf.
 - C2 I was confident I could undo or redo a step if I made a mistake.
 - C3 I understood the consequences of locking a phase before I did it.
 - C4 Waiting for the AI to finish was acceptable.
-

D. Feedback and errors

- D1 When something went wrong, the tool told me clearly what had happened.
 - D2 The messages the tool showed me told me what to do next, not only what failed.
-

E. Fit to real work

- E1 The artefacts the tool produced are ones my team would actually use.
 - E2 The process the tool imposes fits the way my team works.
 - E3 The effort of following the process is worth what it produces.
-

Open questions

1. What was the single most confusing thing about using Apex?
2. Was there a point where you did not trust what the tool had produced? What made you doubt it?
3. If you could change one thing about the interface, what would it be?

Administered in Portuguese to participants selecting that language on G6, using the direct translations in docs/thesis/evaluation/instrument-apex-ux.md: same eighteen items, same five constructs, same three open questions, differing only in wording.

B.5 Interview guides

Two guides, two populations, both evaluating the framework rather than the tool, since SUS, Raw NASA-TLX and the UX questionnaire already cover the tool. Scheduled as a separate 30-45 minute session, not immediately after the task session, and not recorded; the questions below are prompts for a semi-structured conversation, not a script to be read verbatim, and the order and depth of follow-up varied per participant.

Guide 1 - Practitioners

1. Describe how your team currently goes from an idea to code in production, where AI already appears in that (if at all, and who decided that), and what goes wrong most often.
2. The framework splits the lifecycle into six phases with a human decision at the end of each. Which of those decisions would genuinely be made in your team, which rubber-stamped, and which phase would your team resist most?
3. The framework requires a specification to exist before code is generated. What does your team do today instead, what would change, and where would the framework slow you down for no benefit?
4. What would have to be true for your team to adopt this for one project, and what would make you abandon it after two sprints?
5. Who in your team would have to own this for it to survive, and does that person exist?
6. If the tool disappeared tomorrow but you kept the framework, would you still use it? Which parts?
7. When AI writes code that ships and later fails, who is accountable in your organisation today, and does the framework change that?
8. Does the framework's record of what was decided, and by whom, have value to anyone outside the team?
9. What is missing from the framework entirely, and is there anything you expected me to ask and I did not?

Guide 2 - Agile and Scrum experts

Target participants with recognised depth in Agile delivery methodology, not necessarily users of Apex. Given the framework document in advance; this interview concerns the method on paper, not the tool.

1. How would you describe what this framework is, in your own words?
2. Where does this align with Scrum and Agile principles, and where does it depart from them? Is the departure justified by what AI changes, or is it a reversion to stage-gated delivery?

3. The framework has explicit gates between phases. Is that a guardrail or a phase gate in the Waterfall sense, and does the framework avoid the mistake heavyweight processes made in the 2000s?
4. The framework treats roles as hats rather than as people. Does that work in a real team, or does it collapse into one person wearing all of them?
5. Which of the required artefacts would you drop or add, and is the amount of documentation proportionate to what it buys?
6. What kind of team is this framework wrong for, and what would a coach have to teach for a team to run it correctly?
7. Would you recommend this to a client? Under what conditions would you refuse?
8. If you had to defend this framework to a sceptical engineering manager, what would your strongest argument be, and your weakest point?

B.6 Per-participant raw scores

Include the per-participant raw scores in anonymised form - SUS, Raw NASA-TLX by subscale and task, and the UX questionnaire closed items - so the aggregate figures reported in Section 9.4 can be independently recomputed.